
ESTIMACIÓN DE ESTADOS COGNITIVOS EN BASE A ONDAS CEREBRALES.

APLICACIÓN PRÁCTICA CON REDES NEURONALES.



PROYECTO FIN DE CARRERA
PARA LA OBTENCIÓN DEL GRADO EN
INGENIERÍA DE TECNOLOGÍAS INDUSTRIALES

NOMBRE	Paloma Puertas Martínez
TITULACIÓN	Grado en Ingeniería de Tecnologías Industriales
TUTORES	M ^a Guadalupe Sánchez Escribano Gema Fernández-Blanco Martín

INDICE

<i>Relación de Figuras</i>	9
Agradecimientos	11
Resumen	13
Abstract	15
Introducción	17
<i>Marco del Trabajo</i>	20
<i>Objetivos del Trabajo</i>	20
<i>Documento</i>	24
Estado del Arte y de la técnica	25
<i>El lenguaje</i>	28
<i>El pensamiento</i>	31
<i>Recogida de datos cerebrales</i>	34
Herramientas	41
<i>Diadema Mindwave</i>	41
<i>Android Studio</i>	43
<i>R Studio</i>	44
Análisis de datos cognitivos	47
<i>Recogida y preparación de datos</i>	47
<i>Análisis estadístico de los datos</i>	50
Dependencia de las ondas cerebrales con el escenario	50
Dependencia de las ondas cerebrales con la respuesta a las preguntas	56
Modelo de estimación	61
<i>Perceptrón Multicapa</i>	62
Arquitectura	62
Algoritmo de retropropagación	66
<i>Modelo</i>	75
MLP - Una capa oculta	75
MLP - Dos capas ocultas	78

<i>Análisis del Modelo</i>	80
discusión y trabajo futuro	83
Planificación y Programación	87
<i>Estructura de Descomposición del Trabajo</i>	88
<i>Cronograma del Trabajo</i>	89
<i>Presupuesto</i>	90
Anexos	95
<i>Código BCI</i>	95

Relación de Figuras

Figura 1 Usuario utilizando una BCI durante un electroencefalograma. Fuente: (Mérida Perles, 2016-2017)

Figura 2 Libros utilizados durante las sesiones de grabación. Fuente: Elaboración propia

Figura 3 Foto durante una sesión de captura de datos. Fuente: Elaboración propia

Figura 4 Hans Berger, precursor de los estudios con electroencefalograma. Fuente: (Wikipedia, 2017)

Figura 5 Malla flexible para la colocación de los electrodos en la cabeza. Fuente: (Medical EXPO)

Figura 6 Electrodo. Fuente: (Medical EXPO)

Figura 7 Resultado gráfico de un EEG normal. En la primera parte el paciente tiene los ojos cerrados. En la señal señalada (OA), el paciente abre los ojos. Fuente: (Neurodidacta)

Figura 8 Diadema MindWave de Neurosky. Fuente: (MindWave Mobile: User Guide, 2015)

Figura 10 Casco flexible ENOBIO en el que se inserta el número de electrodos correspondiente. Fuente: (Neuroelectronics)

Figura 9 De izquierda a derecha: EPOC+, INSIGHT y EPOC Flex. Fuente: (Emotiv)

Figura 11 Mujer entrenando con los cascos de Halo Sport. Fuente: (Halo Sport)

Figura 12 De izquierda a derecha: Elevvo Medical, Elevvo Wellness y Elevvo Alto Rendimiento. Fuente: (Bitbrain)

Figura 13 STARSTIM, de Neuroelectronics. Fuente: (Neuroelectronics)

Figura 14 Programa R Studio, entorno de desarrollo en R, uno de los lenguajes más utilizados en estadística. Fuente: Elaboración propia

Figura 15 Layout de la BCI desarrollada. Fuente: Elaboración propia

Figura 16 Sala de Seminarios, lugar de realización de las sesiones de grabación

Figura 17 Interior de la sala utilizada para la recogida de datos

Figura 18 Diagrama de cajas de la onda Alfa respecto al Escenario. Fuente: Elaboración propia

Figura 19 Diagrama de cajas de la onda Delta respecto al Escenario. Fuente: Elaboración propia

Figura 20 Diagrama de cajas de la onda Theta respecto al Escenario. Fuente: Elaboración propia

Figura 21 Diagrama de cajas de la onda Beta respecto al Escenario. Fuente: Elaboración propia

Figura 22 Diagrama de cajas de la onda Gamma respecto al Escenario. Fuente: Elaboración propia

Figura 23 Diagrama de cajas de la onda Alfa respecto a la Respuesta. Fuente: Elaboración propia

Figura 24 Diagrama de cajas de la onda Delta respecto a la Respuesta. Fuente: Elaboración

propia

Figura 25 Diagrama de cajas de la onda Theta respecto a la Respuesta. Fuente: Elaboración

propia

Figura 26 Diagrama de cajas de la onda Beta respecto a la Respuesta. Fuente: Elaboración

propia

Figura 27 Diagrama de cajas de la onda Gamma respecto a la Respuesta. Fuente: Elaboración

propia

Figura 28 Arquitectura de una red neuronal tipo con una capa intermedia. Fuente: (Wikipedia, 2018)

Figura 29 Funciones sigmoideal (izquierda) y tangente hiperbólica (derecha). Fuente: (Isasi Viñuela & Galvan, Redes de Neuronas Artificiales: Un Enfoque Práctico, 2004)

Figura 30 Representación del criterio de parada "early stopping". Fuente: (Isasi Viñuela & Galvan, Redes de Neuronas Artificiales: Un Enfoque Práctico, 2004)

Figura 31 Configuración red neuronal con 1 capa oculta de 4 neuronas. Fuente: Elaboración propia

Figura 32 Configuración red neuronal con 2 capas ocultas de 4 y 2 neuronas respectivamente.

Fuente: Elaboración propia

Figura 33 Muestra de 5 resultados de la red de 2 capas ocultas de 4 y 2 neuronas para patrones de entrada pertenecientes al escenario "Vacío"

Figura 34 Porcentaje de acierto para cada escenario con la red neuronal de 1 capa oculta de 4 neuronas. Fuente: Elaboración propia

Figura 35 Porcentaje de acierto para cada escenario con la red neuronal de 2 capas ocultas de 4 y 2 neuronas. Fuente: Elaboración propia

Figura 36 Comparación de la capacidad de clasificación de las dos redes neuronales implementadas. Fuente: Elaboración propia

AGRADECIMIENTOS

Gracias mis tutoras, Gema y Guadalupe, por enseñarme y apoyarme durante la realización de este trabajo, y a Ramón, precursor de este y otros proyectos, cuya ilusión por ellos sigue presente en todos.

Muchas gracias a mis compañeros del portal 4 6H, por amenizarme los últimos años en la ETSII y tener siempre una quiche en el horno.

Por último gracias a mi madre, cuyas clases de ofimática han hecho posible este documento.

RESUMEN

Se presenta el proyecto “Estimación de estados cognitivos en base a ondas cerebrales. Aplicación práctica con redes neuronales”, llevado a cabo en el laboratorio del Centro de Automática y Robótica de la ETSI Industriales y contextualizado dentro del trabajo del Grupo de Control Inteligente.

Enmarcado en el campo de la Inteligencia Artificial, se efectúa un modelado para la clasificación de patrones de la actividad cerebral de una persona en función del estado cognitivo que presenta.

En una primera parte del proyecto, se capturan los datos de la actividad cerebral durante un proceso de comprensión lectora haciendo uso de la diadema Mindwave de Neurosky. Se implementa una aplicación en el entorno de desarrollo Android Studio para la visualización y recogida de los datos, los cuales representan la actividad de las ondas cerebrales. Para la obtención de los valores de las ondas, se han llevado a cabo sesiones de dos tipos de lecturas: una lectura de un texto científico y otra lectura de un texto en el área de la Emoción. Estas sesiones constan de una primera fase de lectura y una segunda fase de preguntas relacionadas con el texto. Como escenario de control se capturan datos de la actividad cerebral de la persona en un escenario en el que no se lee (escenario vacío).

Una vez recogidos los datos, se efectúa un análisis estadístico de los mismos con la herramienta R Studio. Este análisis consiste en un análisis de la varianza (ANOVA) estudiando, por un lado, la influencia del escenario en los valores de las ondas cerebrales y, por otro lado, la influencia de la respuesta a las preguntas (acierto/error) en estos mismos valores.

Tras el estudio de la dependencia de las ondas cerebrales con estos factores, se procede a realizar un modelo matemático para la clasificación de patrones cerebrales en cada uno de los escenarios considerados. Este modelo consiste en una Red Neuronal Perceptrón Multicapa realizada con el programa R Studio.

Del análisis estadístico de los datos capturados se concluye un incremento de actividad de todas las ondas cerebrales en los escenarios de lectura respecto al escenario vacío. Este incremento es mayor con la lectura sobre emociones, lo cual podría deberse a la mayor facilidad para el lector del libro científico por ser de su campo de estudio. Se produce una disminución del nivel de las ondas alpha, delta y theta en las preguntas acertadas respecto a las falladas, mientras que ocurre el caso contrario para las ondas beta y gamma. Estas últimas están asociadas a la atención y concentración.

Los resultados de la prueba de la Red Neuronal muestran que es posible lograr una clasificación de estados cognitivos haciendo uso de redes neuronales y de la diadema Mindwave de Neurosky. Se obtienen, en este caso, mejores resultados con una red neuronal de una capa respecto a una red de dos capas.

Palabras clave: EEG, Inteligencia Artificial, Ciencia Cognitiva, Metodología Experimental, Red Neuronal Perceptrón Multicapa.

ABSTRACT

It is presented the project “Estimación de estados cognitivos en base a ondas cerebrales. Aplicación práctica con redes neuronales”, carried out in the laboratory of the Center of Automation and Robotics located in the ETSI Industriales and contextualized within the work of the Intelligent Control Group.

Framed in the field of Artificial Intelligence, a mathematical model is carried out for the classification of patterns of a person's brain activity based on the cognitive state that he presents.

In the first part of the project, brain activity data is captured during a reading comprehension process using the Mindwave headset of Neurosky. An application is implemented in the Android Studio development environment for the visualization and collection of data, which represent the activity of brain waves. To obtain the values of the brainwaves, sessions of two types of readings have been accomplished: a reading of a scientific text and another reading of a text in the area of Emotion. These sessions consist on a first reading phase and a second phase of questions related to the text. As a control scenario, data of the brain activity of the person is captured in a scenario in which the person is not reading (empty scenario).

Once the data is collected, a statistical analysis is carried out with the R Studio program. This analysis consists of an analysis of the variance (ANOVA) studying, on one hand, the influence of the scenario in the values of the brainwaves and, on the other hand, the influence of the answer to the questions (success / error) on these same values of the waves.

After studying the dependence of brain waves with these factors, we proceed to perform a mathematical model for the classification of brain patterns in each of the scenarios considered. This model consists on a Multilayer Perceptron Neural Network made with the R Studio program.

From the statistical analysis of the captured data, an increase in activity of all brainwaves is recorded in the reading scenarios with respect to the empty scenario. This increase is greater with the reading about emotions, which could be due to the greater facility for the reader to read the scientific book because it is of his branch of study. There is a decrease in the level of alpha, delta and theta waves in the questions answered correctly with respect to the failed ones, while the opposite occurs for the beta and gamma waves. The latter are associated with attention and concentration.

The results of the test of the Neural Network show that it is possible to achieve a classification of cognitive states using neural networks and the Neurosky Mindwave headset. In this case, better results are obtained with a neural network of one layer with respect to a network of two layers.

Keywords: EEG, Artificial Intelligence, Cognitive Science, Experimental Methodology, Multilayer Perceptron Neural Network.

Capítulo 1

INTRODUCCIÓN

El estudio científico de la mente, denominado Ciencia Cognitiva, es un campo interdisciplinario que tiene por objetivo describir y comprender los mecanismos de la cognición, es decir, cómo el cerebro es capaz de aprender. El estudio de las capacidades cognitivas de un modo científico combina la antropología, la física, las matemáticas, las neurociencias, la lingüística, las ciencias computacionales y la Inteligencia Artificial (IA) (Jara-Ettinger). La Ciencia Cognitiva tiene un desarrollo relativamente reciente. A los años noventa se les ha denominado “la década del cerebro” por tener lugar en estos años numerosos estudios con diferentes áreas de aplicación. Es aplicable en toda área donde una persona necesite optimizar funciones del área cognitiva en su interacción con el ecosistema. Resulta de su aplicación una optimización del aprendizaje significativo o el pensamiento crítico, entre otros (Gómez Cumpa, y otros, 2004).

La integración de dichos conocimientos interdisciplinarios, permite dotar a máquinas de capacidades hasta hace poco inimaginables. Se puede dialogar con ellas en entornos controlados, pero también ofrecen nuevos campos tecnológicos y de innovación donde crear soluciones que aporten beneficios al ser humano. En este sentido, las técnicas de neuroimagen y el tratamiento de datos con algoritmos Inteligencia Artificial (IA) ofrecen la posibilidad de estudiar los flujos de actividad eléctrica que se producen en el cerebro en su funcionamiento, y obtener soluciones con aplicaciones desde la industria de los videojuegos a las ciencias de la salud, pasando por multitud de otras muchas que, cada día, aparecen en nuestro entorno tecnológico.

El cerebro muestra cinco formas de onda o formas de propagación del

campo eléctrico en su actividad (Guide, s.f.).

Estas cinco formas de onda se pueden detectar mediante electroencefalógrafos y se clasifican en:

- Delta: Con una frecuencia de onda de 0.5 a 4 Hz las ondas delta son las de mayor amplitud. Se relacionan con el sueño profundo, aunque sin sueños, y la meditación profunda. No se han encontrado indicios de que sean emitidas de forma predominante en un estado de consciencia. A medida que envejecemos las producimos en menor medida. Se relacionan con actividades corporales como la regulación del ritmo cardíaco o la digestión, procesos que no dependen de que se esté en un estado de consciencia.
- Theta: La frecuencia de onda de las ondas Theta es de 4 a 8 Hz. Se relaciona con nuestras capacidades imaginativas, con la reflexión y el sueño. Presenta una elevada actividad cuando experimentamos emociones muy profundas. Por ejemplo, cuando terminamos de hacer algo que nos ha demandado mucha energía y después nos relajamos. Se vincula con fenómenos como la falta de atención, calma profunda o fase REM (en la que soñamos). Se presenta en estados en los que la consciencia está centrada en experiencias imaginativas.
- Alfa: Las ondas alfa tienen una frecuencia de 8 a 12 Hz. Surgen cuando hay calma. Se relacionan con el descanso y el relax, con estados propicios para meditar. Por ejemplo, viendo la tele, en la cama, etc. Un nivel bajo de estas ondas implica estados de ansiedad, estrés o insomnio. Un nivel alto nos impide centrar la atención. Los niños tienden a tener los niveles de ondas alfa mucho más elevados que los adultos.
- Beta: Tienen una frecuencia de 12 a 30 Hz. Las ondas beta se relacionan con estados de atención o vigilia. Estados en los que se está alerta y pendiente de diferentes estímulos. Predominan en momentos en los que se requiere niveles altos de concentración, como durante un análisis de datos, una organización de información, la práctica de deporte, etc.

- Gamma: Con una frecuencia de 30-42 Hz, las ondas gamma son las más rápidas. Se originan en el tálamo y se mueven de la parte posterior hacia delante rápidamente. Se relacionan con tareas de alto procesamiento cognitivo. Por ejemplo, con la tarea de memorizar (Cahn, Levy, Fernandez, & Delorme, 2017).

El estudio de la actividad cerebral se fundamenta en la recepción de las señales cerebrales mediante dispositivos de captación de las mismas para su posterior amplificación y tratamiento de los datos. Para ello, se utiliza una BCI (*Brain-Computer Interface*) con el objetivo, no únicamente de registrar los datos transformando las señales cerebrales en señales de control, sino habitualmente también como medio de visualización de los mismos. Esto resulta especialmente interesante en trabajos en los que se pretende otorgar un feedback al usuario respecto a su propia actividad cerebral.

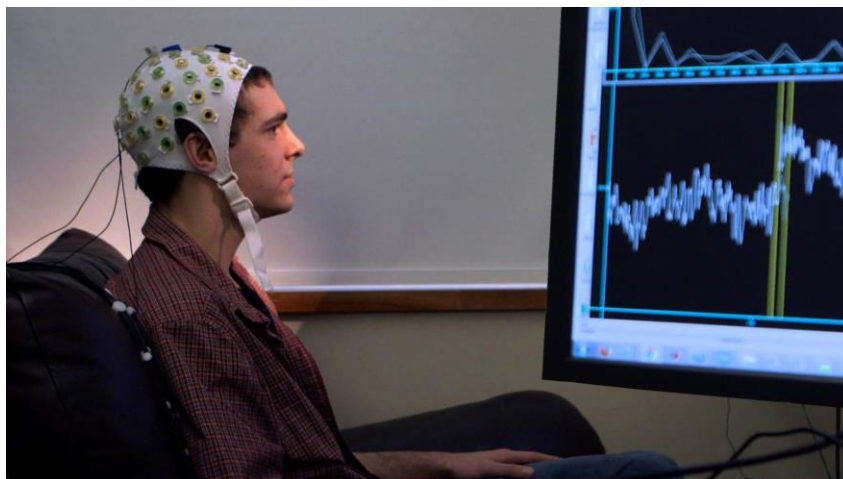


Figura 1 Usuario utilizando una BCI durante un electroencefalograma. Fuente: (Mérida Perles, 2016-2017)

Numerosos trabajos relacionan la actividad cerebral con los diferentes procesos cognitivos del ser humano, lo cual constituye una poderosa herramienta para el tratamiento y diagnóstico médico, entre otros.

El objetivo de este trabajo es la adquisición y posterior análisis de señales EEG (*Electroencefalograma*) mediante un proceso de comprensión lectora. Durante la lectura y comprensión de un texto se ponen de manifiesto todos los procesos cognitivos, por lo que resulta interesante el análisis de la actividad cerebral en la realización de esta tarea.

Marco del Trabajo

En el Laboratorio del Centro de Automática y Robótica se desarrollan tecnologías que se enmarcan en los campos de la ingeniería de control, inteligencia artificial y robótica. El proyecto COGDRIVE (dentro del Grupo de Control Inteligente) lleva a cabo una investigación centrada en asistencia a la conducción autónoma. En este contexto, el proyecto "diadema" tiene como objetivo estudiar los procesos cognitivos asociados a determinadas acciones o patrones conductuales para mejorar la calidad de vida del usuario, siendo sus principales líneas de trabajo: la asistencia a la conducción autónoma ya mencionada (COGDRIVE), salud mental (validación de procesos terapéuticos a través de EEG determinando estados cognitivos) (Martín, Terapia Creativa Inmersiva (T.C.I.)M-000387/2017) y enfermedades asociadas al habla o movimiento tales como la Esclerosis Lateral Amiotrófica (ELA). Este trabajo será continuado e integrado como parte de futuras investigaciones en ámbitos como la conducción autónoma y la salud.

Objetivos del Trabajo

El objetivo general de este trabajo es la realización de un modelo que permita determinar el estado cognitivo de una persona en un momento dado, partiendo de la información que se adquiere a través de la medición de las ondas cerebrales con una diadema Mindwave de Neurosky.

Los objetivos específicos son:

1. Capturar los datos.
2. Integrar los datos.
3. Analizar los datos.
4. Programar una red neuronal.
5. Entrenar la red neuronal programada.

6. Probar la red neuronal.
7. Analizar los resultados.

Para llevar a cabo estos objetivos, el trabajo se subdivide en dos etapas: una fase de captura de datos previa y una fase de modelado.

En la fase de captura de datos se analizarán dos tipos de lecturas diferentes: (a) lectura de un texto científico (“Introducción a la Ciencia. I Ciencias Físicas” de Isaac Asimov) y (b) lectura de un texto en el área de la Emoción (“Qué es una emoción” de David Casacuberta).



Figura 2 Libros utilizados durante las sesiones de grabación. Fuente: Elaboración propia



Figura 3 Foto durante una sesión de captura de datos. Fuente: Elaboración propia

En esta fase se recopilarán los datos de las cinco ondas cerebrales (alfa, beta, gamma, theta y delta) mediante la diadema Mindwave de Neurosky

para realizar un análisis de los mismos con el programa de análisis de datos.

Tras la fase de lectura se presenta una fase de preguntas (preguntas de verdadero/falso, de desarrollo y de tipo test con varias opciones). Las respuestas son registradas tanto por su acierto/error como por el tiempo dedicado a cada pregunta. Todo este proceso es capturado por la diadema Mindwave de Neurosky.

La fase de modelado consiste en la implementación de una red neuronal siendo testada posteriormente. De forma concreta este trabajo tiene los siguientes objetivos definidos:

I. Fase de toma de Datos

- a) Captura de datos: El objetivo de la primera parte de esta fase es la recogida de datos de la actividad cerebral en 3 escenarios diferentes, realizando 5 sesiones de recogida por cada escenario. Dos de estos escenarios se corresponden con los dos tipos de lecturas que se van a realizar. El tercer escenario se corresponde con el escenario vacío, en el que no se lee ningún texto, actuando este de control. Para la recogida de datos se desarrolla una BCI en el entorno de desarrollo Android Studio. Cada sesión de recogida de información consta de dos fases:
 - a. Una primera parte de lectura del texto, en la que se registran los valores de las ondas cerebrales mientras la persona lee un texto o, en el caso del escenario vacío, mientras permanece sentada con los ojos abiertos.
 - b. Una segunda parte de preguntas sobre el texto que se ha leído anteriormente.
- b) Integración de datos: En esta parte se almacenan y organizan los datos en formatos de modelado con el objetivo de viabilizar su tratamiento con el programa de análisis de datos. El objetivo es disponer de dos bases de datos: una con los datos recogidos de la actividad cerebral y el escenario al que pertenece cada conjunto de

datos, y una segunda base de datos con los valores registrados de las ondas cerebrales durante la parte de preguntas y el resultado de la pregunta, es decir, si fue acertada o errada.

- c) **Análisis de datos:** Se realiza un análisis estadístico de los datos con la finalidad de estudiar la dependencia del estado cognitivo de una persona con los escenarios propuestos, así como valorar la influencia del conocimiento o desconocimiento de la respuesta a las preguntas en la actividad cerebral.

II. Fase de Modelado

- a) **Programación de MLP (*MultiLayer Perceptron*) en R:** Se diseña una Red neuronal Perceptrón multicapa con el programa de análisis estadístico R. Se hace uso del paquete “neuralnet” para su diseño e implementación.
- b) **Entrenamiento de la Red:** En la fase de entrenamiento se envían muestras de los datos de las ondas cerebrales a través de la Red neuronal indicándosele a qué escenario pertenece cada muestra o patrón. De esta manera la Red aprende a reconocer los patrones cerebrales.
- c) **Prueba de la Red:** Finalizado su entrenamiento, la Red está preparada para clasificar los patrones que se le introduzcan como entrada. En esta fase se comprueba la capacidad que ha adquirido la Red neuronal como clasificadora de estados cognitivos.
- d) **Análisis de Resultados:** Por último se analiza la calidad de la Red neuronal implementada comparando las clasificaciones que realiza con el escenario real al que pertenece cada patrón.

Previamente a la recogida de datos y modelado, se realiza un estudio del estado del arte. Se investigan los trabajos realizados anteriormente enfocados al estudio de los estados cognitivos, centrando el examen en aquellos que hacen uso de los electroencefalogramas como medio de

visualización de estos estados. Se presenta un análisis de los dispositivos que se encuentran disponibles en el mercado relacionados con el estudio de la actividad cerebral.

Documento

El documento está estructurado en cuatro bloques o partes diferentes:

- Un bloque de introducción en el que se presentan los trabajos y estudios que se han realizado anteriormente así como las diferentes tecnologías que se pueden emplear en trabajos de este campo.
- Un bloque en el que se presenta el trabajo técnico realizado al igual que las herramientas utilizadas para el mismo.
- Un bloque de conclusiones sobre los resultados obtenidos en el trabajo y posibles líneas futuras de aplicación de los mismos.
- Un bloque de dirección del proyecto donde se incluyen los documentos de la metodología PMI aplicados al proyecto realizado.

Capítulo 2

ESTADO DEL ARTE Y DE LA TÉCNICA

Una forma de abordar el estudio de ciertos patrones cerebrales consiste en registrar el comportamiento de las ondas que emiten. Estas ondas se llaman ondas cerebrales. Fundamentalmente el cerebro emite 5 tipos de ondas cerebrales: alfa, beta, theta, delta y gamma. Estas ondas son el resultado de la actividad eléctrica del cerebro, cada una de estas ondas está asociada a diferentes estados mentales.

La onda alfa está relacionada con la falta de actividad cerebral (Blasi & Roig). Es una onda con una frecuencia de entre 8-12 Hz. Cuando una persona toma un descanso después de una actividad o cuando medita o da un paseo, se originan ondas alfa. Por otro lado, cuando el cerebro está activo y concentrado en actividades mentales, genera las ondas beta. Este tipo de ondas son de una amplitud más baja y una frecuencia de entre 12-30 Hz. Por ejemplo, durante una conversación intensa o un debate, o dando una clase un profesor, se genera la onda beta. En cambio, cuando un individuo está tan relajado que comienza a soñar despierto, se emiten las ondas theta. Estas ondas surgen, por ejemplo, cuando se lleva un rato conduciendo y resulta imposible recordar los últimos kilómetros recorridos, es decir, durante esas actividades que realizamos automáticamente, como ducharse. Las ondas theta tienen una frecuencia de entre 4-8 Hz. Este estado mental es muy positivo para la generación de ideas. Por otra parte, las ondas delta se originan en el momento de sueño profundo. Son las ondas de menor frecuencia, entre 0,5-4 Hz. Cuando se apaga la luz y se cierran los ojos para dormir, las ondas cerebrales descienden en frecuencia hasta la onda delta, en la que se alcanza el sueño profundo (Herrmann, 1990). La onda gamma es la

más rápida, con frecuencias de 30-42 Hz. Esta onda a menudo se ha señalado como especialmente relevante para la conciencia (Yuval-Greenberg, Tomer, Keren, Nelken, & Deouell, 2008) y relacionada con experiencias de meditación (Braboszcz, Cahn, Levy, Fernandez, & Delorme, 2017).

Dependiendo del nivel de actividad una onda cerebral será predominante sobre el resto, sin embargo, el resto de ondas también estarán presentes. Así, aunque una persona se encuentre en un estado activo y su onda predominante sea la onda beta, también estará emitiendo el resto de ondas aunque sea a un nivel bajo (Herrmann, 1990). Analizaremos por tanto la presencia de las 5 ondas cerebrales durante un proceso de lectura.

La actividad de leer involucra diferentes fases mentales denominadas procesos cognitivos (Vygotsky, 1995). Cada uno de estos procesos supone una acción cerebral diferente que interviene en el momento de la lectura. Se hace necesario, por tanto, comenzar explicando en qué consisten estos mecanismos cerebrales.

Los procesos cognitivos son estructuras o mecanismos mentales (Banyard, y otros, 1995) que se ponen en funcionamiento cuando el hombre observa, lee, escucha o mira.

Estos procesos son: percepción, atención, pensamiento, memoria y lenguaje.

De entre estos procesos cognitivos, se caracterizan como básicos aquellos que se pueden producir sin la intervención consciente del sujeto y tienen una raíz biológica (Viramonte, 2000), lo cual no implica que el sujeto no pueda lograr un control sobre ellos. Son aquellos que permiten que la información sea captada y almacenada en nuestro sistema para así poder procesarla y reaccionar ante ella. Estos procesos cognitivos básicos son la atención, la percepción y la memoria.

La atención es la capacidad de concentración espontánea o voluntaria de la conciencia en un objeto o estímulo que puede ser externo o interno. La atención supone filtrar las distracciones y concentrarse en la información verdaderamente relevante. Engloba, a su vez, tres fenómenos como son: la

alerta, que es la respuesta comportamental fisiológica a la entrada de estímulos o bien la receptividad incrementada a estos; la selección, que implica una categorización de los estímulos; y la activación, preparación o disposición psicológica para la acción (Vallejo Ruiloba, 2015). La atención juega un rol fundamental para la percepción, la cognición y la coordinación motora, ubicándose en la base de la cognición humana tanto por su importancia como por su relevancia como mediadora en otros procesos cognitivos (Carboni & Barg, 2016). Se asocia a un estado de atención el incremento de actividad de las ondas beta y gamma mientras que la onda alfa se asocia a estados de falta de concentración (Sanei & Chambers, 2013).

La percepción es el proceso mediante el cual se interpreta la información que se ha recibido a través de los sentidos. Involucra la decodificación cerebral y la significación de la información que se recibe. De esta manera se puede operar con ella o almacenarla. Por ejemplo, la comprensión de un texto leído no involucra únicamente el acto de decodificar signos, sino también la interpretación de las relaciones entre ellos.

Como último proceso cognitivo básico se encontraría la memoria. Se entiende por memoria la capacidad de retener y evocar información de naturaleza perceptual o conceptual (Viramonte, 2000). Esto es, es la facultad para retener y recordar el pasado así como para almacenar conocimiento e interpretaciones sobre el mismo. Cuando se memoriza, en primer lugar, se codifica la información a alguna clase de representación mental (acústica, visual, semántica...) (Banyard, y otros, 1995). A continuación, se almacena durante un periodo de tiempo para, posteriormente, recuperarla. La codificación de la información puede implicar la conexión con otra información ya memorizada, es decir, es un proceso activo.

El pensamiento y el lenguaje no se consideran procesos cognitivos básicos ya que requieren una intervención consciente del sujeto. Pensar es un proceso cognitivo a través del cual se crean, regulan y modifican las ideas. Este proceso mental se encarga de tareas relativas a la resolución de problemas, el razonamiento, la toma de decisiones, etc. En cuanto al

lenguaje, los seres humanos desarrollamos la capacidad de expresión de nuestros sentimientos o pensamientos a través de este sistema de comunicación estructurado. Mediante la palabra transmitimos ideas para comunicarnos con nuestro entorno.

Al ser estos dos procesos cognitivos realizados de manera consciente, resulta interesante desarrollar de manera más detallada el impacto de ambos en la actividad mental. A continuación se presentan algunos resultados de diferentes investigaciones relacionadas con el lenguaje y el pensamiento, que tratan de mostrar qué ocurre en el cerebro durante un proceso cognitivo de habla, lectura, y otras actividades que intervienen en la comprensión lectora.

2.1. El lenguaje

En el campo de la salud mental, las afecciones del lenguaje suponen una parte dominante debido a las múltiples enfermedades que afectan al habla directamente (afasias) o indirectamente (enfermedades que afectan al sistema nervioso principalmente). Con un amplio campo de actuación en este sector, el análisis del cerebro en su relación con el uso del lenguaje es de gran interés y son muchos los estudios realizados en esta línea.

El *síndrome de Landau-Kleffner* (SLK) es un trastorno infantil que afecta al lenguaje que se caracteriza por la pérdida de la capacidad para utilizar o comprender el lenguaje oral. Su diagnóstico pasa por la realización de un EEG, ya que el EEG de un niño afectado por este trastorno muestra anomalías, en la fase de sueño lento principalmente (Nieto Barrera, 2005). Otra afección del lenguaje que muestra también diferencias en el EEG medido durante la fase de sueño es la Disfasia de desarrollo (Picard, y otros, 2008). Este es un trastorno grave del desarrollo del lenguaje que se da sin que existan trastornos asociados, es por tanto necesario la realización de un diagnóstico diferencial, ya que a menudo se confunde con otras patologías como autismo o sordera (Ruiz del Castillo, García Mateo, Abendea Alonso, & Segovia Pintos, s.f.). En el caso del autismo, el trastorno lleva asociada

una incapacidad para la interacción social y la comunicación. Esta anormalidad en la acción comunicativa se traduce en un comportamiento diferente de las ondas cerebrales respecto a un cerebro sano (Benítez-Burraco & Murphy, 2018).

Además de los trastornos de afección directa al lenguaje, otras enfermedades tienen consecuencias en la capacidad de comunicación de una persona. La Esclerosis Lateral Amiotrófica (ELA) afecta al sistema motor principalmente y también produce una discapacidad en el habla y la comprensión del lenguaje. Gracias al entrenamiento con un EEG, enseñando a los pacientes a alcanzar determinados estados mentales, se puede llegar a un control de herramientas que ayuden a la comunicación (Birbaumer, y otros, 2000). No únicamente en el área del lenguaje, dentro del campo de la salud se encuentran numerosas aplicaciones de los EEG. Como avance en la epilepsia, por ejemplo, gracias a un EEG se ha logrado predecir los ataques epilépticos con una antelación de 7 minutos de media (Le Van Quyen, y otros, 2001).

Debido a los numerosos trastornos y enfermedades con afección en el lenguaje, se hace notoria la necesidad de analizar mediante un electroencefalograma cómo actúa el cerebro en relación con el lenguaje.

Para análisis de la actividad cerebral relacionado con el lenguaje es de utilidad estudiar los patrones cerebrales en función de las palabras pronunciadas por una persona. Esta clasificación de las palabras pronunciadas por un sujeto en función de sus ondas cerebrales pasa por la transformación de dichas ondas en palabras. Para ello se pueden utilizar diferentes métodos de transformación de las ondas a palabras: la transformación en sonidos (*sonificación*) y en texto (*textificación*) para, tras la transformación, mediante el uso de un clasificador, tratar de clasificar correctamente la palabra que el usuario ha pronunciado. Con el uso de la sonificación se obtienen mejores resultados, ya que resulta más acertada que la textificación como método de transformación de las ondas cerebrales en lenguaje (González-Castañeda, Torres-García, Reyes-García, & Villaseñor-Pineda, 2016). De hecho, aplicando este método de transformación, la

sonificación, se logra una capacidad de clasificación de lenguaje no hablado superior a la clasificación sin transformación previa a sonidos de las señales cerebrales (González-Castañeda E. F., Torres-García, Reyes-García, & Villaseñor-Pineda, 2014).

Profundizado en el uso del lenguaje, además de en forma de sonidos y texto, el ser humano utiliza las palabras en sus pensamientos. Durante un proceso imaginativo, la mente puede vagar en imágenes o en texto. El análisis de las ondas cerebrales en cada uno de estos casos muestra que, cuando la mente vaga en imágenes, se observa un mayor incremento de la onda alfa que cuando vaga en forma de lenguaje. La onda alfa está relacionada con estados de calma (Villena-González, López, & Rodríguez, 2016), con lo que se concluye que el pensamiento involucrando el lenguaje, como por ejemplo durante un diálogo interior, conlleva una mayor activación. De hecho, los datos de la actividad cerebral obtenida con un electroencefalograma, contienen información que hace posible la clasificación de las palabras no pronunciadas, como se ha estudiado con las palabras “arriba” y “abajo” del idioma español (Torres García, Reyes García, & Villaseñor Pineda, 2011) , entre otros estudios de la misma línea de investigación.

Gracias a los EEG, se muestra que es posible la predicción de las vocales y consonantes en palabras habladas o imaginadas (Xiao-Mei & Jeremy Hill, 2012). Además, esta decodificación es posible sobre expresiones individuales sin el requerimiento de que las señales cerebrales se promedien de entre cientos de repeticiones, como se venía haciendo en estudios cerebrales más tradicionales. Esto es una señal alentadora de que puede ser factible en un futuro próximo decodificar palabras completas en tiempo real, permitiendo así a los usuarios comunicarse en silencio, sin la necesidad de movimiento muscular. No obstante, la comunicación sin la pronunciación de palabras es una práctica muy común ya que, aunque a menudo sin ser consciente, una persona mantiene un diálogo interior consigo misma muy frecuentemente.

Vygotsky (1896-1934), psicólogo ruso dedicado especialmente a la psicología del desarrollo, concibió el lenguaje interior como una forma de

habla privada que desempeña una función cognitiva y de dominio del propio comportamiento (auto reguladora). Además, considera que el lenguaje interior desempeña el papel de factor contribuyente en la transición del pensamiento al lenguaje en voz alta (Vygotsky, 1995).

El estudio de los procesos de diálogo interior tiene particular interés en relación a los estudios de la imaginación en esquizofrenia, la cual está ligada a las alucinaciones auditivas con actividad en las zonas del lenguaje. Durante un diálogo interior, se produce un aumento de actividad cerebral entorno al giro inferior frontal izquierdo, situado junto a la sien (McGuire, y otros, 1996). La lectura de un texto está estrechamente relacionada con el diálogo interior ya que, al leer, se reproducen mentalmente las palabras del mismo.

2.2. El pensamiento

La lectura incluye, en su proceso de comprensión de la misma, el reconocimiento de las palabras y su asociación con conceptos almacenados en la memoria, el desarrollo de las ideas significativas, y la relación entre lo que se lee y lo que ya se sabe. Todo ello se realiza repetidamente para poder alcanzar la comprensión del texto. La lectura demanda prestar atención activa a muchas cosas al mismo tiempo para coordinar los procesos psicológicos que se dan en la comprensión de textos escritos. Por ello, a menudo, la mente pierde la atención en lo que se lee y vaga en otros pensamientos.

El momento en el que la mente vaga (en inglés “*mind-wandering*”), se considera un estado de pensamiento independiente del estímulo (“*Stimulus Independent Thought*”, *SIT*). Como su nombre indica, en esta situación los pensamientos surgen independientemente del estímulo que se pueda estar recibiendo, es decir, se produce un desacoplamiento que lleva a la atenuación de los estímulos externos (Schooler, y otros, 2011). Un pensamiento de este tipo que surja durante la lectura de un texto llevaría a la desconexión con los estímulos externos (en este caso el texto que se lee) y, por tanto, a una falta de asimilación del contenido del mismo.

Cerca del 50% de los pensamientos surgen independientemente de los estímulos que la persona recibe (Killingsworth & Gilbert, 2010). Como resulta evidente, en su relación con la comprensión lectora, se observa una correlación negativa entre el estado en que la mente vaga y la comprensión del texto (Schooler, Reichle, & Halpern, 2004).

Manteniendo la concentración en el texto, la mente reproduce imágenes o visualiza lo que se lee (Llorente, 2016), con lo cual cabe preguntarse la influencia de esta imaginación en la actividad cerebral. Analizando la banda de frecuencia Beta de una persona durante la imaginación de objetos, se observa un predominio de la actividad prefrontal con tendencia derecha, con compromiso del lóbulo frontal derecho y del lóbulo parietal derecho. Por el contrario, durante la resolución de cálculos matemáticos se encuentra un predominio de actividad prefrontal con tendencia izquierda (Sánchez, y otros, 2017).

Además de en la imaginación de objetos, se han realizado numerosos estudios relacionados con la imaginación de movimiento debido al campo de aplicación que esto conlleva en la línea de la robotización de prótesis. Un ejemplo de ello es la observación de la Desincronización Relacionada con el Evento (DRE) que se produce durante la imaginación de movimiento de las manos derecha e izquierda. La DRE hace referencia al fenómeno de atenuación de la amplitud de determinadas ondas ante la aparición de un evento. Con el movimiento o la imaginación del movimiento se produce una DRE en las ondas alfa y beta. Se observa una diferencia en la DRE de las ondas alfa y beta en función de si se imagina el movimiento con la mano derecha o la izquierda (Pfurtscheller, Neuper, Flotzinger, & Pregenzer, 1997).

De la misma manera, la anticipación de un movimiento propio pero inducido por un robot, es decir, la previsión de que un miembro del propio cuerpo se va a mover por una fuerza externa, genera igualmente una DRE (Norman, y otros, 2016).

Como se ha visto, la imaginación de objetos y movimiento de una persona es analizable mediante los resultados de un EEG. Esto nos da una idea de la

posibilidad de diferenciación de pensamientos imaginativos mediante el estudio de las ondas cerebrales.

Una combinación de la imaginación de objetos y de movimiento podría ser la imaginación de un objeto en movimiento. Esta línea de investigación tiene una aplicación muy amplia si suponemos el movimiento de un cursor en una pantalla. Mediante entrenamiento, midiendo las ondas cerebrales y dando feedback al sujeto que se entrena, se puede lograr el control de un cursor mediante el pensamiento (Kostov & Polak, 2000).

Más allá de las imaginaciones que se generan en la mente, las emociones evocadas por la lectura tienen un impacto en las ondas cerebrales que se emiten. Un estado en el que se siente alguna emoción, frente a un estado neutro, provoca una activación del lóbulo frontal con un incremento de actividad de la onda beta tanto en el lóbulo frontal como en el parietal. En el caso de comparar una emoción de felicidad con una de tristeza, se observan diferencias en el comportamiento de la onda alfa, la cual presenta mayor coherencia en determinados electrodos cuando se está feliz. (Choppin, 2000). Por tanto, se manifiesta efectivamente una influencia del estado emocional del individuo en el comportamiento de sus ondas cerebrales.

Vygotsky afirmaba que el pensamiento y el lenguaje tenían raíces genéticas distintas pero que se influían de manera recíproca. Establecía una interrelación entre ambas capacidades en determinados momentos del desarrollo de una persona. De esta manera, consideraba la existencia de etapas de habla pre-intelectuales así como de pensamiento pre-lingüístico. Sugería que el desarrollo de una persona pasaba, no tanto por la evolución de ambas funciones, como por la evolución de la relación y las conexiones entre ellas.

En este trabajo medimos las ondas cerebrales de una persona mediante un único canal (electrodo). Sin embargo, hay numerosos dispositivos para la adquisición de datos con variaciones en el número de canales de recogida de señales cerebrales.

2.3. Recogida de datos cerebrales

En la medida de los estados cognitivos se hace necesario medir la actividad cerebral en forma de datos numéricos. Fue en 1920 cuando comenzó a estudiarse la actividad cerebral en humanos. El doctor alemán Hans Berger fue quien los empezó, demostrando en 1929 que la actividad eléctrica de los cerebros dormidos y despiertos es diferente, lo cual supuso una revolución en el campo de la neurología y la psiquiatría.

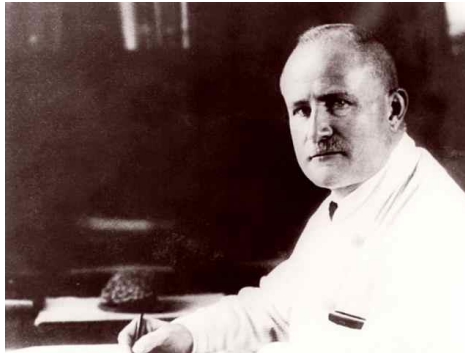


Figura 4 Hans Berger, precursor de los estudios con electroencefalograma. Fuente: (Wikipedia, 2017)

Habitualmente se recogen los valores de las ondas cerebrales como representación del estado cognitivo de una persona. Para la recopilación de esta información, en la actualidad podemos encontrar numerosos dispositivos de medida con diferentes propósitos de uso de la información.

Para aplicaciones que requieren un nivel de precisión elevado, como en el ámbito sanitario, se recurre a colocar un mayor número de electrodos y realizar un electroencefalograma (*EEG*) detallado, recogiendo datos de diferentes zonas del cerebro. Un electroencefalograma es un examen que sirve para medir la actividad eléctrica del cerebro. Consiste en registrar mediante electrodos las corrientes eléctricas que se forman en las neuronas cerebrales, y que son la base del funcionamiento del sistema nervioso. Para la sujeción de todos los electrodos se coloca un casco que cubre la cabeza y sustenta los receptores.

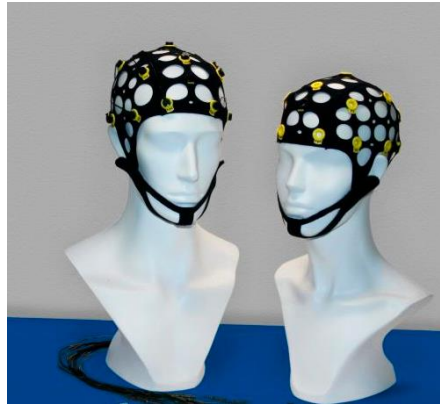


Figura 5 Malla flexible para la colocación de los electrodos en la cabeza. Fuente: (Medical EXPO)

Hay varios fabricantes de cascos EEG. Habitualmente están formados por una tela flexible con forma adaptable a la cabeza, en la que se encuentran los electrodos y los cables que los conectan.



Figura 6 Electrodo. Fuente: (Medical EXPO)

Mediante un EEG se pueden detectar enfermedades nerviosas como la epilepsia, la narcolepsia o demencias, entre otras.

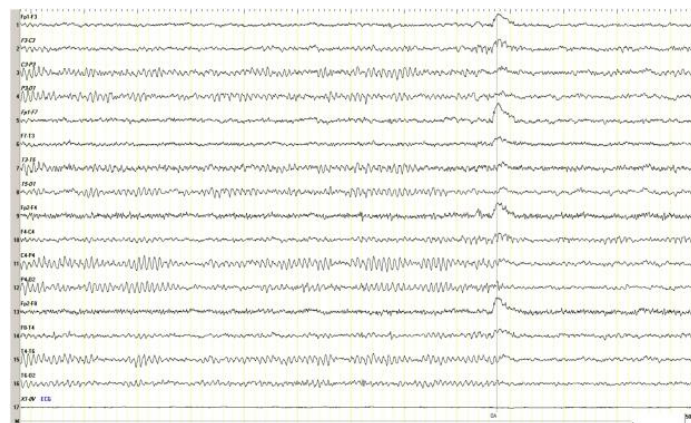


Figura 7 Resultado gráfico de un EEG normal. En la primera parte el paciente tiene los ojos cerrados. En la señal señalada (OA), el paciente abre los ojos. Fuente: (Neurodidacta)

Varias empresas se han apoyado en el mercado del EEG para el desarrollo y comercialización de productos relacionados con el análisis de la actividad cerebral.

Uno de los dispositivos disponibles es la diadema MindWave de Neurosky. Debido a su alta disponibilidad por su económico precio se utiliza por numerosos programadores de aplicaciones. Se han desarrollado aplicaciones para su uso como BCI, como juegos que incorporan la concentración o la relajación para jugar.

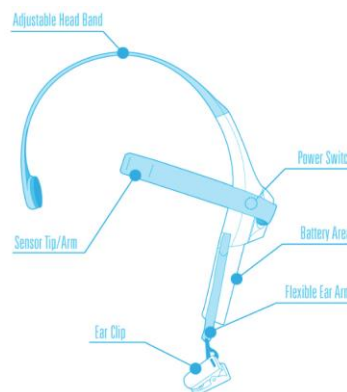


Figura 8 Diadema MindWave de Neurosky. Fuente: (MindWave Mobile: User Guide, 2015)

Esta diadema cuenta con un electrodo que se coloca en la frente y otro de referencia que se ajusta al lóbulo de la oreja izquierda. Tiene la capacidad de medir los valores de 5 ondas cerebrales: delta (0,5 a 4 Hz), theta (4 a 8 Hz), alfa (8 a 12 Hz), beta (12 a 30 Hz) y gamma (30 Hz en adelante), así como el nivel de atención y relajación. Adicionalmente, dispone de detección de pestañeo. La diadema MindWave de Neurosky es un dispositivo de bajo coste viable para los estudios de la actividad cerebral. Anteriormente se ha demostrado la capacidad del dispositivo para detectar la aparición del potencial P300, el cual se genera cuando el sujeto recibe un estímulo de interés, pero poco frecuente, de entre otros irrelevantes pero frecuentes. Aunque el nivel de precisión con esta diadema es inferior al de otros dispositivos con un mayor número de canales de recogida de señales, su disponibilidad y bajo coste justifican su utilización en este tipo de trabajos (Vivar Vera, y otros, 2017).



Figura 9 Casco flexible ENOBIO en el que se inserta el número de electrodos correspondiente.
Fuente: (Neuroelectrics)

Además del potencial P300, la diadema es capaz de discriminar estados de concentración respecto a los de relajación, así como reconocer la presencia o ausencia de dolor en un sujeto en reposo, basándose en el registro de la onda Beta del mismo (Torres, Sánchez, & Palacio-Baus, 2014). Al igual que Neurosky, hay otras empresas que desarrollan productos para realizar EEG:

La empresa Emotiv tiene tres productos hardware diferentes para realizar un EEG. Los comercializan para optimizar la capacidad cerebral del usuario, controlar dispositivos con la mente o para su uso en investigación, entre otros. Por un lado, cuentan con la diadema EPOC+, orientada a investigación, con 14 sensores EEG; por otro INSIGHT, orientado al bienestar y a la interfaz hombre-ordenador, con 5 sensores EEG; y por último EPOC Flex para investigación profesional, con una malla ajustable y



Figura 10 De izquierda a derecha: EPOC+, INSIGHT y EPOC Flex. Fuente: (Emotiv)

hasta 32 sensores EEG configurables.

Por otro lado, otra empresa del sector es Neuroelectrics. Esta compañía tiene numerosos productos destinados al estudio del cerebro. Para la lectura de datos disponen de ENOBIO, un casco con 8 (ENOBIO 8), 20 (ENOBIO 20)

o 32 (ENOBIO 32) sensores en función de la variante.

ENOBIO 8 está destinado a usarse fuera del laboratorio ya que consta de una configuración sencilla. Está preparado igualmente para investigación o uso clínico. ENOBIO 20 está destinado a aplicaciones que requieran una mayor recogida de datos, como por ejemplo estudios de sueño. ENOBIO 32 se utiliza en actividades que requieren la recogida de un gran volumen de datos, por ejemplo uso clínico y telemedicina.

Hemos mostrado algunos ejemplos de dispositivos destinados al registro de indicadores de la actividad del cerebro para la posterior utilización de esta información. Sin embargo, hay productos utilizados para otras actividades, como la estimulación cerebral. Se trata de dispositivos que provocan un determinado estado de actividad en el cerebro.

Un ejemplo de ello es Halo Sport. Halo Sport se presenta en forma de cascos. Estimulan la parte del cerebro responsable del movimiento de los músculos, de este modo, acelera la mejora en la fuerza, resistencia y habilidad en el deporte y en otros aspectos de la vida. Está basado en el hecho de que la señal que envía el cerebro para mover un músculo cuando se entrena, se toca un instrumento musical, etc. es una señal compleja. Es el envío de estas señales, el cerebro tiene que perfeccionar la señal que envía al músculo, y por ello, mediante Halo Sport, se estimula esa función cerebral, logrando así una mayor eficiencia en el proceso.



Figura 11 Mujer entrenando con los cascos de Halo Sport. Fuente: (Halo Sport)

Otra empresa con este tipo de productos es BitBrain, que presenta como

dispositivo para la estimulación cerebral Elevvo. Lo comercializan en forma de 3 paquetes distintos. Elevvo Medical, para su uso médico en pacientes con enfermedades que cursan con deterioro cognitivo. Se utiliza en pacientes con Trastornos Depresivos, de Déficit de Atención e Hiperactividad, entre otras. Consta de un casco (malla) de 8/16/32 sensores y un software base. Elevvo Wellness, destinado a la población general que busque mejorar sus capacidades cognitivas o un envejecimiento saludable. Consta de un casco con 12 sensores y un software base. Elevvo Alto Rendimiento, orientado a profesionales que requieran un alto rendimiento cognitivo (deportistas, militares, cuerpos especiales...). Consta de un casco con 12 sensores y un software base.

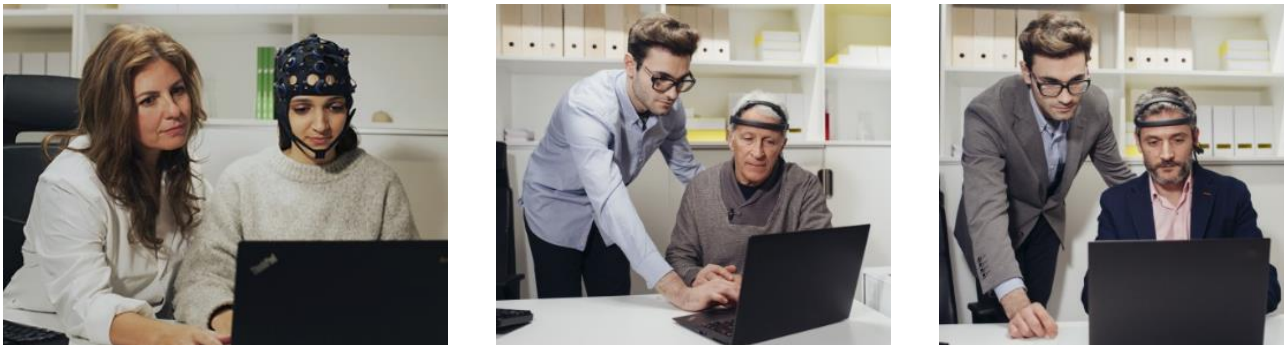


Figura 12 De izquierda a derecha: Elevvo Medical, Elevvo Wellness y Elevvo Alto Rendimiento. Fuente: (Bitbrain)

Se ha presentado anteriormente la empresa Neuroelectrics y sus productos para la realización de EEG, pero también dispone de un producto para la estimulación cerebral, STARSTIM.



Figura 13 STARSTIM, de Neuroelectrics. Fuente: (Neuroelectrics)

Este dispositivo tiene una malla de neopreno ajustable a la cabeza con

posiciones para la inserción de electrodos. Se comercializan las opciones STARSTIM 8, 20 y 32, en función de los electrodos insertados. Se utiliza para tratar trastornos como la depresión, así como para la mejora de procesos cognitivos

Capítulo 3

HERRAMIENTAS

En este capítulo se presentan las diferentes herramientas utilizadas para la ejecución del trabajo. Se ha hecho uso de entornos de programación como Android Studio, para el desarrollo de aplicaciones Android, o R Studio, siendo este último una potentísima herramienta de análisis de datos con un paquete para el modelado de redes neuronales. Además, se ha empleado un dispositivo para realizar EEG, pieza fundamental para la recepción de las ondas cerebrales.

3.1. Diadema Mindwave

Por su disponibilidad y demostrada capacidad para los estudios de este tipo, en este proyecto se va a utilizar la diadema de la compañía Neurosky con el fin de registrar y posteriormente analizar datos que reflejen la actividad del cerebro. La diadema Mindwave Mobile 2 de Neurosky consta de 2 sensores. Uno de referencia en un clip que se ajusta en el lóbulo de la oreja izquierda, y otro en un brazo ajustable a la frente. Esta diadema registra valores de atención y meditación en un rango de 0 a 100 en función de un algoritmo de medida. A mayor valor, mayor nivel de atención o meditación. También tiene la capacidad de detectar los pestañeos y de medir valores de actividad de las ondas cerebrales.

Con la diadema de Neurosky, los valores de las ondas cerebrales se miden en decibelios y representan la densidad espectral de potencia de las 5 bandas de frecuencia de las ondas. La densidad espectral de potencia es la representación de la potencia de una señal distinguiendo el contenido de los

distintos armónicos, es decir, describe su contenido en potencias en función de la frecuencia. Algunas de sus características son:

- Usa el módulo TGAM1
- Emparejamiento inalámbrico automático
- Batería AAA individual
- 8 horas de funcionamiento de la batería
- ID de diadema estático (las diademas tienen una identificación única para fines de emparejamiento)
- Plataformas compatibles: Windows (XP / 7/8/10), Mac (OSX 10.8 o posterior), iOS (iOS 8 o posterior) y Android (Android 2.3 o posterior)

Medidas:

- Salidas de ondas cerebrales de 12 bits (3 - 100Hz) con frecuencia de muestreo a 512Hz
- Salidas de espectros de potencia EEG (Alpha, Beta, etc.)
- Salida de un medidor de eSense patentado por NeuroSky, como Atención, Meditación y otros.
- Análisis de calidad de la señal EEG / ECG (se puede usar para detectar un contacto deficiente o si el dispositivo está fuera de la cabeza)

Características físicas:

- Peso: 90g
- Con el brazo sensor arriba: altura: 225mm x ancho: 155mm x grosor: 92mm
- Con el brazo sensor abajo: altura: 225mm x ancho: 155mm x grosor: 165mm

Conexión Bluetooth:

- Módulo de modo dual BT / BLE: BT (SPP) para PC, Mac, Android, BLE (GATT) para iOS
- Rango BT: 10m

Esta diadema es utilizada tanto para juegos (con un gran número de aplicaciones Android y iOS disponibles) como para desarrolladores, principalmente trabajando con Arduino.

Para el desarrollo de investigaciones con la diadema Neurosky proporciona

de manera gratuita el código en java de una aplicación Android en la que se hace uso de las distintas funciones aplicables con la diadema y que forman parte de su librería. Esta aplicación de ejemplo se puede descargar y ejecutar en un dispositivo Android. El paquete “*Android Developer tools 4.2*” se encuentra en la página web de Neurosky.

3.1. Android Studio

Para el desarrollo de la BCI (*Brain-Computer Interface*) se ha utilizado el entorno Android Studio. Este es un entorno de desarrollo de Android utilizado para la creación de aplicaciones ejecutables en dispositivos Android.

Android Studio está basado en el software IntelliJ IDEA de JetBrains y fue lanzado como reemplazo a Eclipse como el IDE oficial para el desarrollo de aplicaciones para Android. Esta aplicación ofrece una herramienta completa para desarrollar y depurar aplicaciones para el sistema operativo de Google para dispositivos móviles.

Con ella podremos realizar la edición de código, depuración, utilizar herramientas de rendimiento, y cuenta con un sistema flexible de compilación y creación y despliegue instantáneo, que le permite centrarse en la creación de aplicaciones.

Android Studio incluye plantillas de proyecto y de código que hacen que sea fácil para añadir patrones bien establecidos, tales como un panel lateral de navegación y la vista de páginas. Es posible iniciar el proyecto con una plantilla de código o incluso hacer clic derecho en una API en el editor y seleccionar “buscar código de ejemplo” para buscar ejemplos.

Dado que se han creado numerosas aplicaciones para trabajar con Neurosky, e incluso se pueden encontrar ejemplos de utilización de su librería, se ha recurrido a este entorno de desarrollo para trabajar con la diadema.

Se ha modificado la aplicación de ejemplo descargada de Neurosky añadiendo la escritura de los datos recogidos a un fichero de texto así como

unos botones para el registro de tiempo de las preguntas realizadas después de la lectura.

3.2. R Studio

Se ha utilizado el programa R Studio para realizar los análisis estadísticos y la red neuronal. Se trata de un entorno gratuito que los desarrolladores pueden descargar. En este entorno de desarrollo se programa en el lenguaje R, uno de los lenguajes más utilizados en estadística.

El lenguaje R proporciona una gran gama de herramientas estadísticas entre las que se incluyen el análisis de datos y la generación de gráficos. Este lenguaje tiene capacidad de generar gráficos de una elevada calidad.

El proyecto R fue iniciado en 1995 por Ross Ihaka y Robert Gentleman del Departamento de Estadística de la Universidad de Auckland. R es un dialecto de S, lenguaje desarrollado en 1976 por John Chambers. R es un lenguaje Orientado a Objetos, lo cual lo hace simple y flexible. R se distribuye gratuitamente bajo la licencia GNU GPL (General Public Licence). Es compatible con Mac, Linux y Windows.

Aunque el uso habitual de R es desde su propia consola en R Studio, algunas empresas ponen a disposición de los algunos de los IDEs, ejecutándose el código de la aplicación en el propio momento, obteniéndose informes, visualizando gráficos obtenidos, etc.

R proporciona una amplia gama de herramientas estadísticas (tests estadísticos, modelos lineales y no lineales, análisis de series temporales, algoritmos de clasificación y agrupamiento, etc.) y gráficas de gran detalle. R también puede usarse como herramienta de cálculo numérico, área en la que puede ser tan eficaz como otras herramientas específicas para ello como son GNU Octave o MATLAB.

R Studio es un entorno muy utilizado en Big Data por su capacidad de manejo de grandes volumen de datos. Además, se pueden importar los datos desde otras plataformas como Excel, ficheros de texto, SPSS, etc. Además,

al ser R ampliamente utilizado, con cerca de 7000 paquetes disponibles, existe una gran comunidad de usuarios gracias a los cuales se pueden encontrar soluciones a los problemas más habituales que pueden surgir durante el trabajo en este entorno de programación y con este lenguaje. Estas características lo convierten en una potente herramienta de cálculo.

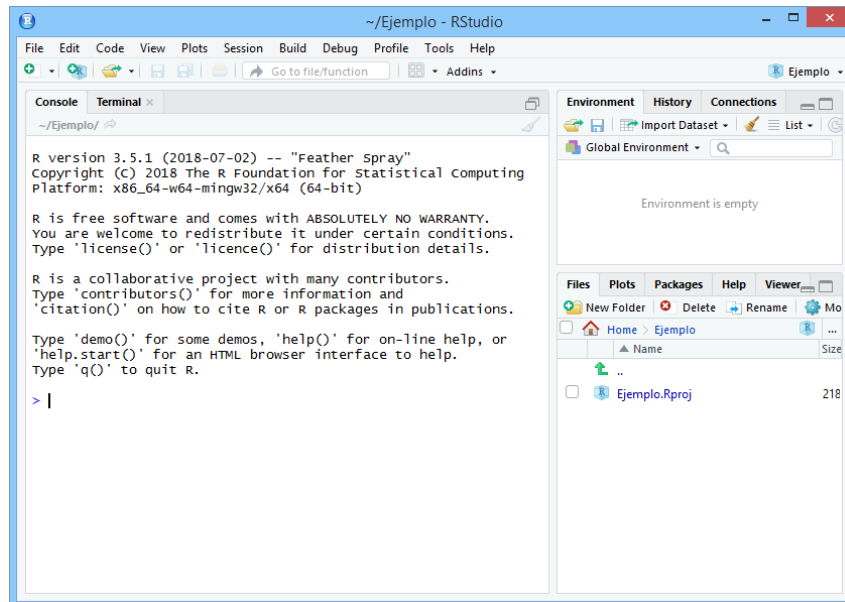


Figura 14 Programa R Studio, entorno de desarrollo en R, uno de los lenguajes más utilizados en estadística. Fuente: Elaboración propia

Para el análisis de los datos registrados con la diadema Mindwave se utilizará R Studio debido al gran volumen de datos almacenado y su capacidad de manejo de los mismos. Se importarán los datos desde Excel, ordenados en columnas y se hará, en primer lugar, un análisis estadístico de la varianza. Además se obtendrán gráficos para facilitar la visualización de los análisis realizados.

Posteriormente, se descargará el paquete “neuralnet” de R para el desarrollo y entrenamiento de una red neuronal. Este paquete facilita el diseño, entrenamiento y testeo de las redes neuronales que se quieran desarrollar. Entre las funciones disponibles dentro del paquete “neuralnet” la más destacable es la función de entrenamiento de la red (neuralnet), que permite la especificación de parámetros como el número de capas ocultas, la función de activación o el método de parada del entrenamiento.

Capítulo 4

ANÁLISIS DE DATOS COGNITIVOS

En el análisis del estudio de la mente y sus estados cognitivos, la primera parte consiste en la obtención de datos que reflejen la actividad del cerebro. Con este fin, se encuentran en el mercado numerosos dispositivos con la capacidad de recoger y amplificar las señales cerebrales para su almacenamiento o utilización del usuario. Una vez recogidos estos datos se procede a su interpretación mediante un análisis estadístico, en este caso un análisis de la varianza, a partir del cual se visualiza la relevancia que ciertos factores tienen en determinados patrones cerebrales.

4.1 Recogida y preparación de datos

Para la recogida de datos se ha desarrollado una aplicación Android utilizando como base la aplicación ejemplo de Neurosky. Esta aplicación de ejemplo, además de conectar con la diadema, muestra en una gráfica de líneas la evolución temporal de los valores de las 5 ondas cerebrales que recoge la diadema MindWave.

Para su adecuación al uso que se le da en este trabajo, se ha añadido la funcionalidad de escritura de los datos en un fichero de texto, así como unos botones para registrar el tiempo destinado a cada pregunta en los escenarios de lectura. Para mayor información, se muestran también por pantalla los valores numéricos que toman las ondas.

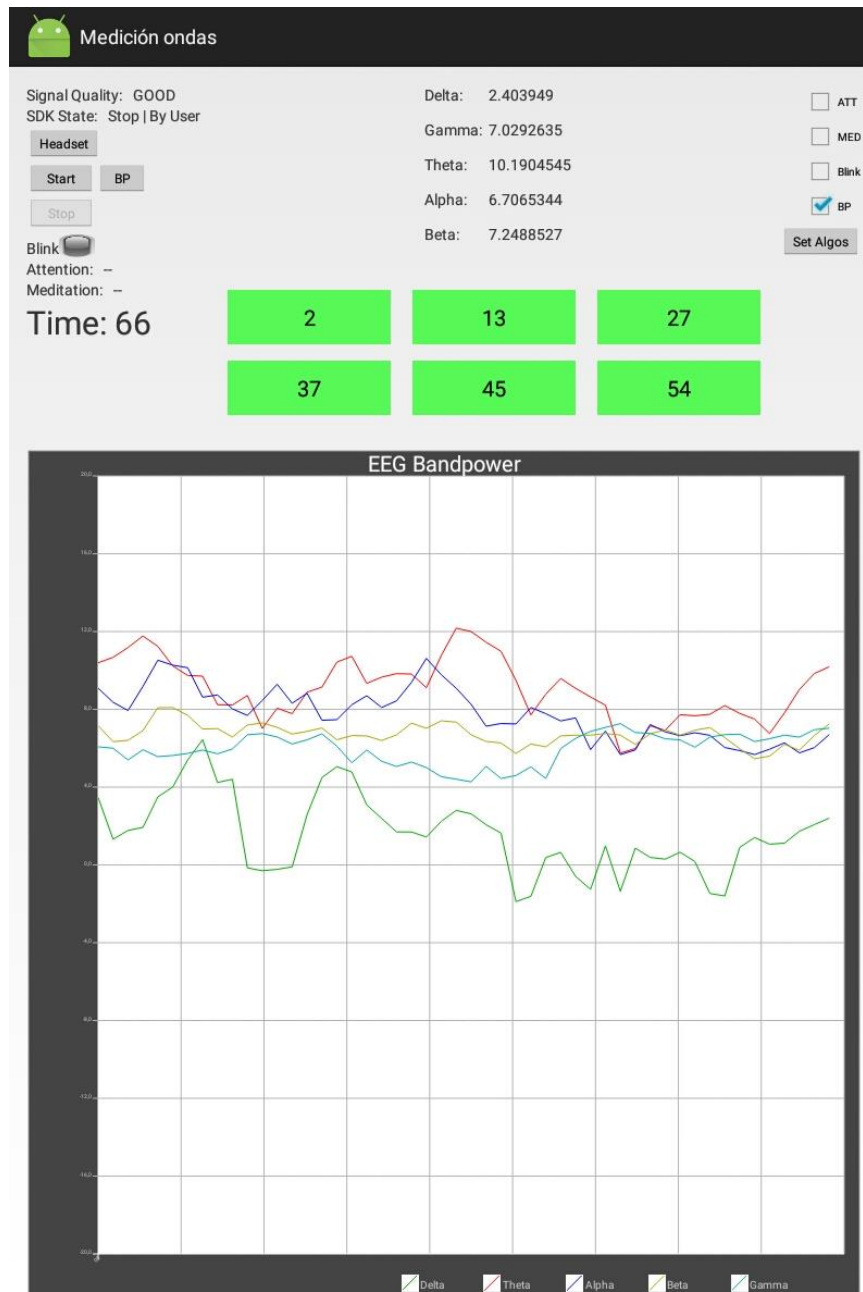


Figura 15 Layout de la BCI desarrollada. Fuente: Elaboración propia

Se han recogido los datos en 5 sesiones por cada escenario a evaluar. En el escenario vacío el sujeto no realiza ninguna acción pero tiene los ojos abiertos. Se recogen los datos durante 3 minutos por sesión. En el escenario científico el individuo (sujeto experimental) lee un texto de ciencia durante aproximadamente 3 minutos y, a continuación, se le somete a 6 preguntas sobre el texto leído. Estas 2 fases de la sesión se registran en archivos diferentes para facilitar su posterior análisis. En el escenario de emociones

se lee un texto sobre psicología y se procede de la misma manera que con la lectura científica, esto es, 5 sesiones con cada parte, una parte de lectura y otra de preguntas.

Para controlar la recogida de datos se escoge siempre la misma sala de la Escuela, a las mismas horas y con la misma persona tomando los datos (sujeto investigador/ psicóloga). Las preguntas se seleccionan previamente y son divididas en 3 tipos (preguntas de verdadero/falso, de desarrollo y de tipo test con varias opciones).



Figura 16 Sala de Seminarios, lugar de realización de las sesiones de grabación



Figura 17 Interior de la sala utilizada para la recogida de datos

Posteriormente a la recogida de los datos, se realiza un análisis estadístico de los mismos. Para ello, es necesaria la preparación de los ficheros en Excel de forma que se trabaje con ellos en R Studio.

Se prepara un fichero que contiene 6 columnas: el escenario y las 5 ondas cerebrales. En la columna del escenario se encuentran los valores que toma

esta variable, esto es, 0 para el escenario vacío, 1 para el científico y 2 para el de emociones. En las columnas de las ondas se hallan los datos que la diadema ha registrado para cada una de las ondas durante el proceso de lectura.

Paralelamente se genera otro archivo Excel de 6 columnas en las que están, por un lado, los datos de las 5 ondas cerebrales registrados durante el proceso de preguntas y, en la sexta columna, los datos que toma la variable “respuesta”. Esto es, 1 para las preguntas acertadas, 0 para las falladas. Con la preparación de estos dos ficheros se puede proceder a realizar un análisis estadístico de los datos.

4.2 Análisis estadístico de los datos

Para analizar la influencia de la comprensión de la lectura en la actividad cerebral, se va a realizar un análisis estadístico con el programa R. Se va a comenzar con un análisis de la varianza de los datos registrados durante la lectura, para determinar si el factor “Escenario” es significativo en los valores que toma cada una de las 5 ondas cerebrales registradas. El factor “Escenario” toma, por tanto, 3 posibles valores: Escenario Vacío, Escenario de lectura Científica y Escenario de lectura sobre Emociones.

Posteriormente se estudian los datos registrados durante el proceso de preguntas sobre la lectura leída. Se pretende determinar si el factor “Respuesta”, que toma valores de acertada o errónea, resulta significativo en los valores de las ondas cerebrales.

Dependencia de las ondas cerebrales con el escenario

Se comienza estudiando si el escenario en el que se encuentra el sujeto cuyas ondas se evalúan es determinante en los datos registrados, es decir, si la actividad de leer un libro u otro, o no leer, influye de manera significativa en las ondas cerebrales.

Para evaluar el nivel de dependencia de las ondas con el escenario, se realiza un análisis de la varianza o ANOVA (*ANalysis Of VAriance*). Este análisis va a determinar si las medias de los valores de las ondas varían al variar el escenario de manera significativa. Para ello, compara las medias de las variables de respuesta, en este caso las ondas cerebrales, en los diferentes niveles de los factores, en este caso un factor (Escenario) con 3 niveles (vacío, científico y emociones).

El método para probar la hipótesis de igualdad de medias, conocida como “hipótesis nula”, consiste en obtener un estadístico, llamado F, que representa el grado de similitud existente entre las medias que se están comparando. En este estudio trataría de comparar el parecido entre las medias de los valores de las ondas cerebrales en cada uno de los escenarios que se evalúan.

Para el cálculo del estadístico F se procede de la siguiente manera:

- En primer lugar se calcula la suma de cuadrados de la diferencia de cada observación y la media total, que representa la variabilidad total, es la Suma de Cuadrados Total. El número de grados de libertad es el número de casos total menos 1: $n - 1$.

$$SC_T = \sum (x_i - \bar{x})^2$$

- A continuación se calcula la suma de las diferencias de la media para cada escenario y la media global, multiplicada por el número de muestras en cada escenario. Este término es la Suma de Cuadrados del Factor. El número de grados de libertad es el número de grupos menos 1: $k - 1$.

$$SC_F = \sum n_K (\bar{x}_K - \bar{x})^2$$

- Posteriormente se calcula la Suma de Cuadrados Residual, la suma de cuadrados de las desviaciones entre cada dato y la media de su grupo. El número de grados de libertad es el número total de casos menos el número de grupos: $n - k$.

$$SC_R = \sum (x_{ik} - \bar{x}_K)^2$$

- Se calcula a continuación las medias cuadráticas. En el análisis de la varianza, una media cuadrática es el cociente entre la suma de cuadrados y sus correspondientes grados de libertad.

$$MC_T = \frac{SC_T}{n-1}; \quad MC_F = \frac{SC_F}{k-1}; \quad MC_R = \frac{SC_R}{n-1}$$

- Por último, se calcula el valor del estadístico F. Este valor informa sobre la viabilidad de aceptar o rechazar la hipótesis nula de igualdad de medias.

$$F = \frac{MC_F}{MC_R}$$

Con el fin de visualizar los resultados de un análisis de la varianza, se realiza una tabla con la información más relevante utilizada para el análisis. Habitualmente tiene la siguiente forma:

FUENTE DE VARIACIÓN	SUMA DE CUADRADOS (SC)	GRADOS DE LIBERTAD (gl)	MEDIA DE CUADRADOS (MC)	F calculado
FACTOR	SC Factor	k -1	SC Factor/ k-1	MC Factor/MC Residual
RESIDUAL	SC Residual	n -k	SC Residual/ n-k	
TOTAL	SC TOTAL	n -1		

Tabla 1 Presentación tabla ANOVA

En este punto, en análisis de la varianza está finalizado, pero falta determinar si el valor F calculado acepta o rechaza la hipótesis nula. Para comprobar esto, se compara el valor calculado con un valor determinado que depende de los grados de libertad del factor y de los residuos y también del valor α , que es la confianza con la que se puede aceptar o rechazar la hipótesis. Este valor con el que se compara se encuentra tabulado. Se rechazará la hipótesis nula si el valor F calculado es mayor o igual que el valor tabulado.

La herramienta R Studio permite realizar este análisis con el gran volumen de datos que se maneja en este caso. Como resultado, muestra el factor que se ha evaluado, en este problema el escenario, con 2 grados de libertad, y el

nivel de significación de este factor en cada onda cerebral.

A modo de ejemplo se muestra la salida de la consola de R Studio para el resumen del ANOVA para la onda alfa:

Df	Sum Sq	Mean Sq	F value	Pr(>F)
Escenario	2	1755	877.3	595.1 <2e-16 ***
Residuals	3262	4808	1.5	
---Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1				

La tabla muestra el factor que se está valorando, en este caso el Escenario, con sus grados de libertad, la suma de cuadrados, la media de cuadrados y el valor F calculado. Así como los grados de libertad de los residuos, y su suma y media de cuadrados correspondiente.

El valor F calculado es 595,1 mientras que el valor tabulado para estos grados de libertad y un α de 0,95 es de 2^{-16} . En este caso es claramente rechazable la hipótesis de igualdad de medias. Es decir, podemos rechazar la hipótesis de que la onda alfa mantiene una igualdad de medias en los escenarios, por tanto, el escenario es un factor significativo en los valores que toma la onda alfa.

Procediendo de igual manera para el resto de ondas cerebrales, el análisis concluye que el factor “Escenario” es significativo en todas las ondas. Para la visualización de este hecho se representa mediante un diagrama de cajas la relación del factor con cada una de las variables, es decir, con cada una de las ondas cerebrales.

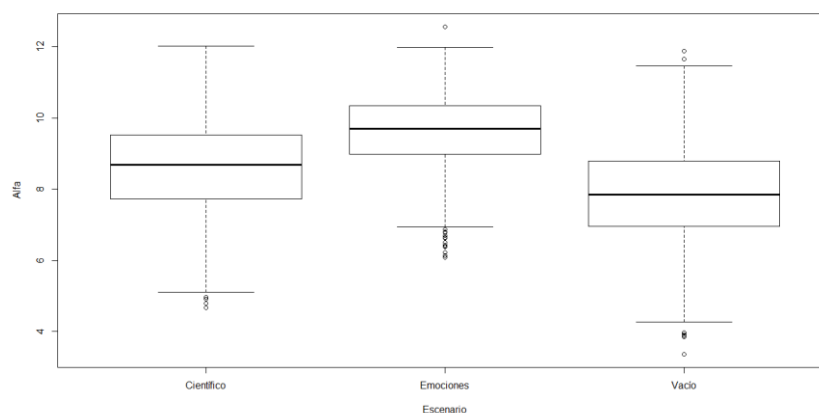


Figura 18 Diagrama de cajas de la onda Alfa respecto al Escenario. Fuente: Elaboración propia

Como se muestra en la figura 18, la onda Alfa tiene unos valores superiores en el escenario de lectura sobre emociones respecto a los obtenidos durante la lectura científica. El nivel de la onda aumenta en ambos escenarios de lectura respecto al escenario vacío.

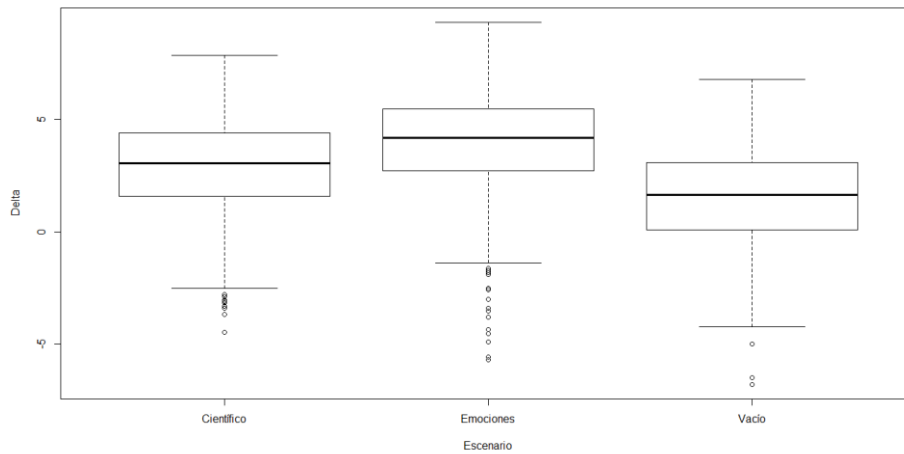


Figura 19 Diagrama de cajas de la onda Delta respecto al Escenario. Fuente: Elaboración propia

En el caso de la onda Delta, esta también toma valores superiores en el escenario de lectura sobre emociones respecto a los obtenidos durante la lectura científica. El nivel de la onda aumenta en los escenarios de lectura frente al escenario vacío.

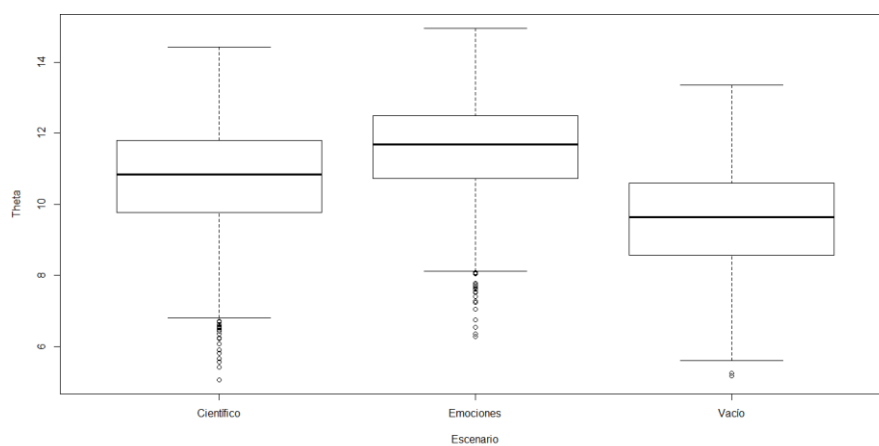


Figura 20 Diagrama de cajas de la onda Theta respecto al Escenario. Fuente: Elaboración propia

La onda theta, se presenta de igual manera, mostrando una actividad mayor

en la lectura de emociones frente a la lectura científica. El nivel de la onda aumenta en ambos escenarios de lectura respecto al escenario vacío.

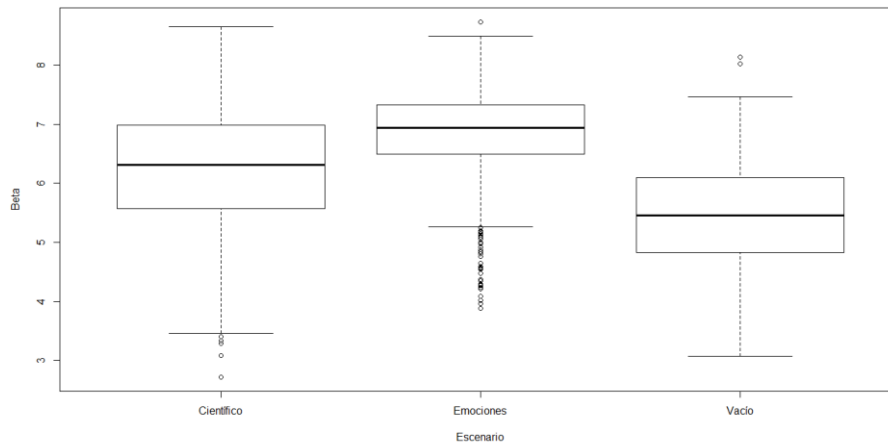


Figura 21 Diagrama de cajas de la onda Beta respecto al Escenario. Fuente: Elaboración propia

En el caso de la onda beta, además de observar el mismo resultado en cuanto a los valores que toma la onda en cada escenario, podemos ver menos variabilidad en el escenario sobre emociones (la caja es más estrecha).

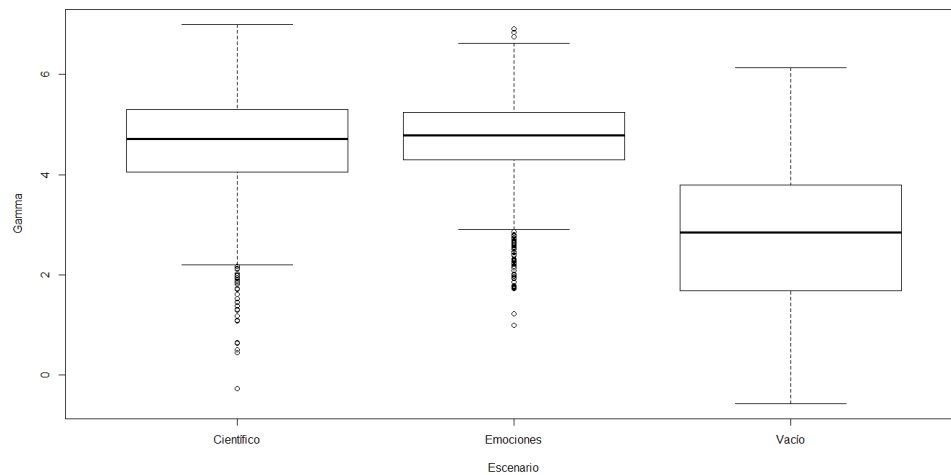


Figura 22 Diagrama de cajas de la onda Gamma respecto al Escenario. Fuente: Elaboración propia

Por último, la onda gamma presenta unos resultados ligeramente diferentes al resto de ondas cerebrales. En su caso, los resultados para cada escenario de lectura son muy parecidos. El nivel de la onda theta, al igual que en el

resto, aumenta para las lecturas respecto al escenario vacío.

Se puede apreciar que efectivamente hay una influencia del escenario en las ondas cerebrales.

Dependencia de las ondas cerebrales con la respuesta a las preguntas

A continuación se plantea el análisis de si la actividad cerebral es diferente en función de si se acierta o se falla una pregunta sobre el texto que se ha leído. A priori parece que el esfuerzo mental cuando se falla una pregunta será mayor, ya que primero se hace un esfuerzo por recordar lo que se ha leído y supone mayor dificultad elaborar una respuesta cuando no se está seguro de si es correcta o no. Sin embargo, cuando se conoce la respuesta a una pregunta que se plantea, parecería que una persona se encuentra más relajada.

Para comprobar, a partir de los datos que se tienen, cómo influye realmente la respuesta a las preguntas en las ondas cerebrales, se realiza un análisis de la varianza en el que el factor es “Respuesta”. Se le ha asignado los valores de puntuación “0” cuando se trataba de una respuesta incorrecta, y “1” cuando la respuesta era acertada.

Se procede de igual manera que en el caso anterior. Se parte de un fichero en que se encuentran los valores de las cinco ondas cerebrales y, asignado a cada patrón de valores, el valor correspondiente del factor “Respuesta”.

A modo de ejemplo, se presenta la tabla que muestra R Studio como resultado del análisis de la influencia del factor “Respuesta” en la variable “Alfa”, es decir, el estudio de si acertar o fallar una pregunta es significativo en los valores que toma esta onda. El resultado es el siguiente:

	Df	SumSq	MeanSq	F value	Pr(>F)
Respuesta	1	39.1	39.08	18.63	1.77e-05 ***
Residuals	860	1803.7	2.10		
---Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1					

A la vista del resultado, se concluye que la onda alfa es dependiente de la

respuesta a la pregunta realizada. Procediendo de igual manera para el resto de ondas cerebrales, obtenemos el mismo resultado para las ondas theta, delta y gamma. Sin embargo, la onda beta no presenta una dependencia significativa con el resultado de la pregunta, siendo la tabla resultado de su análisis de la varianza la siguiente:

	Df	Sum Sq	Mean Sq	F value	Pr(>F)
Respuesta	1	2.8	2.801	2.12	0.146
Residuals	860	1135.8	1.321		

La onda beta presenta un comportamiento diferente al resto de ondas. Se muestran a continuación los diagramas de cajas correspondientes de cada una de las ondas respecto a la respuesta.

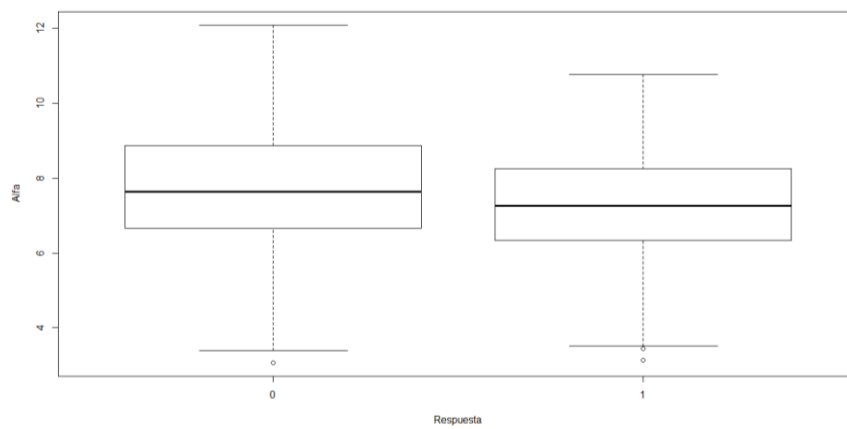


Figura 23 Diagrama de cajas de la onda Alfa respecto a la Respuesta. Fuente: Elaboración propia

La onda alfa tiene un nivel de actividad superior en las preguntas contestadas erróneamente respecto al nivel que toma en las preguntas acertadas. Se observa que el intervalo de valores que se abarca en las preguntas erradas es mayor, es decir, hay más variabilidad de resultados en este tipo de respuestas.

La onda delta tiene unos resultados parecidos. En este caso también el nivel de actividad es superior en las preguntas contestadas erróneamente respecto al nivel que toma en las preguntas acertadas.

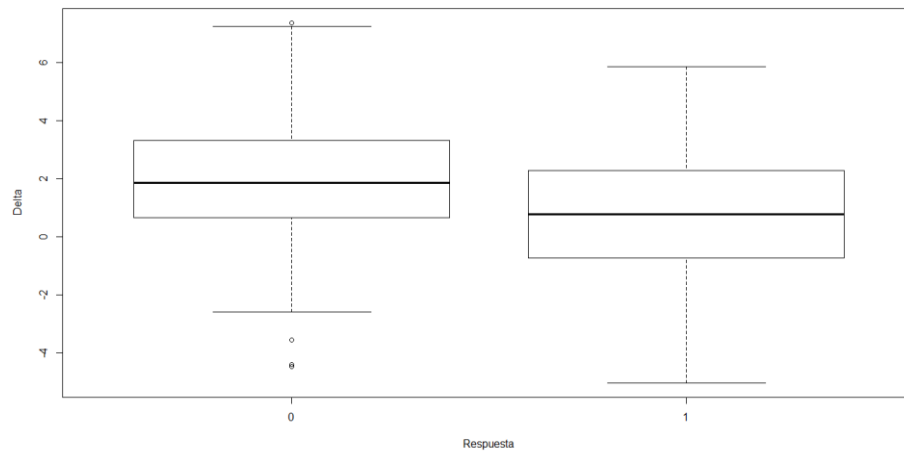


Figura 24 Diagrama de cajas de la onda Delta respecto a la Respuesta. Fuente: Elaboración propia

Se observa que el intervalo de valores que se abarca en ambos tipos de preguntas es muy similar, a diferencia de lo que ocurre con la onda alfa.

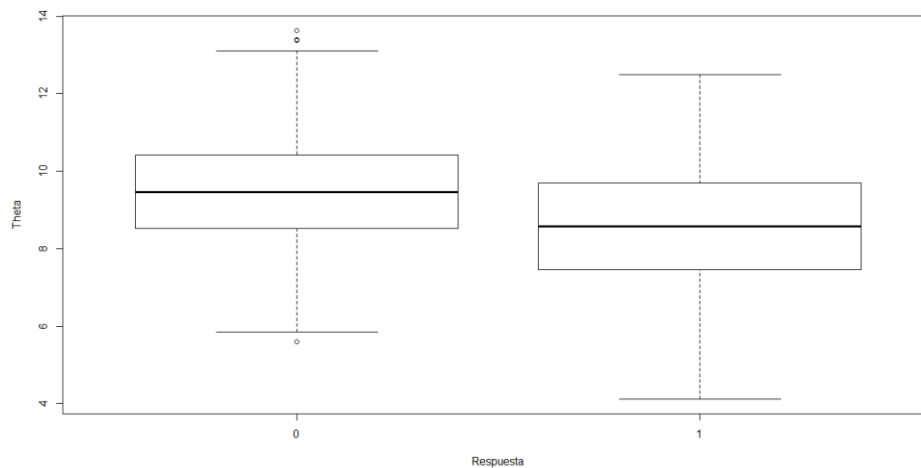


Figura 25 Diagrama de cajas de la onda Theta respecto a la Respuesta. Fuente: Elaboración propia

En el caso de la onda theta, se observa el mismo tipo de resultados en cuanto a en qué escenario los valores son superiores. Igualmente el nivel de actividad es superior en las preguntas contestadas erróneamente respecto al nivel que toma en las preguntas acertadas. En cuanto al intervalo que toman los resultados, este es similar en ambos escenarios.

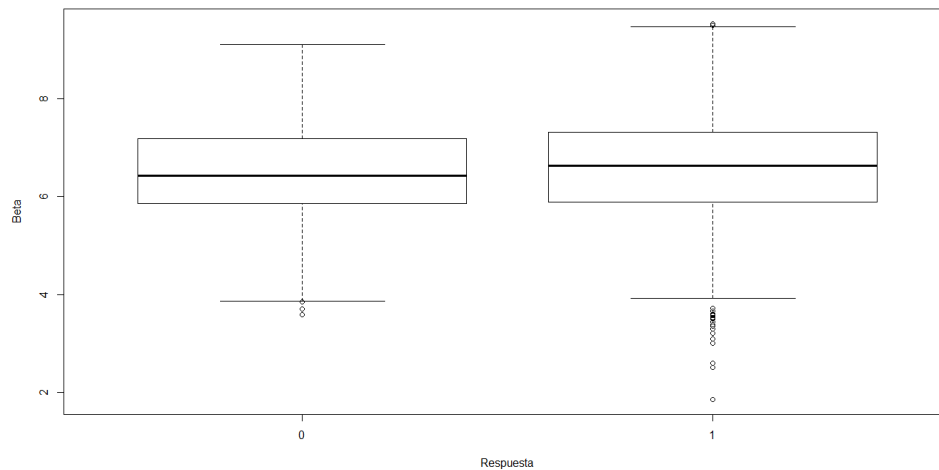


Figura 26 Diagrama de cajas de la onda Beta respecto a la Respuesta. Fuente: Elaboración propia

El caso de la onda beta es diferente al resto de ondas por la falta de significación del escenario en el valor que toma la onda. En el diagrama de cajas se puede observar una media superior en el caso de las preguntas acertadas respecto a las preguntas erradas.

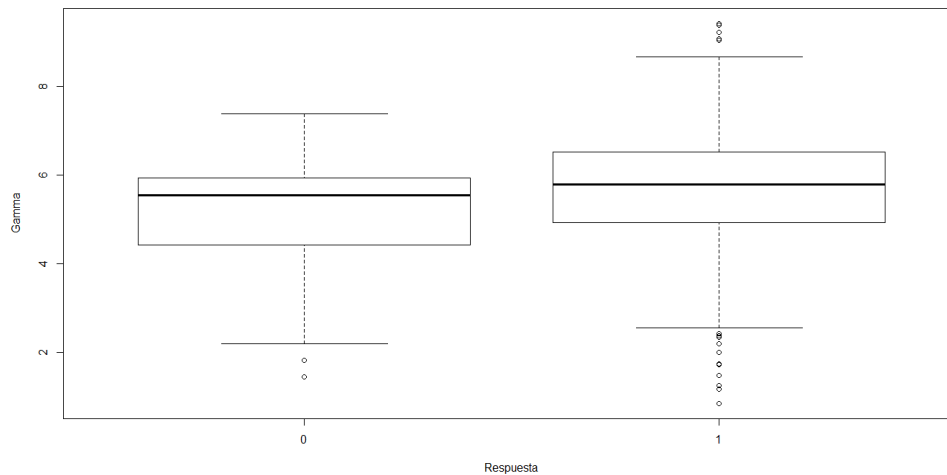


Figura 27 Diagrama de cajas de la onda Gamma respecto a la Respuesta. Fuente: Elaboración propia

Por último, la onda gamma refleja, al igual que la onda beta, un incremento de nivel de actividad en el caso de las preguntas acertadas respecto a las erradas. En este caso el escenario, según los resultados obtenidos con el ANOVA si es un factor significativo en los valores que toma la onda. En las preguntas erradas se observa que la variabilidad de los resultados por debajo

de la media es superior a la de los resultados por encima de la media.

Capítulo 5

MODELO DE ESTIMACIÓN

Las redes neuronales toman este nombre porque son, en esencia, simulaciones de cómo funciona un cerebro típico, pero programado en una computadora. Son una manera de que una máquina pueda aprender, detectar patrones y tomar decisiones en torno a ello, tal y como lo haría cualquier ser humano.

Habitualmente un programa actuaría esperando recibir comandos para a continuación actuar. Sin embargo, una red neuronal toma una enorme cantidad de información previa, que son los “patrones de entrenamiento” y utiliza esta información para inferir una respuesta por su cuenta. Si el programador, o entrenador, nota algún error, se lo hace saber. De esta forma la red neuronal puede adoptar este nuevo conocimiento, tomarlo como un resultado negativo y así, mejorar la precisión de información generada en el futuro. Es decir, las redes neuronales están en constante aprendizaje, alimentándose de la información que se le proporciona.

En los problemas en los que se pretende que, a partir de un determinado patrón, la Red efectúe una clasificación del mismo, es habitual el uso de la conocida como Red neuronal Perceptrón Multicapa, o MLP (*MultiLayer Perceptron*). Las limitaciones de las redes de una sola capa hicieron que se plantease la necesidad de diseñar redes en las que se aumentase el número de capas, esto es, introducir capas intermediarias o capas ocultas entre la capa de entrada y la capa de salida de forma que se pudiese implementar cualquier función con el grado de precisión deseado, es decir, que las redes multicapa fuesen aproximadores universales.

Perceptrón Multicapa

Un Perceptrón multicapa surge, como se ha comentado, por la limitación que tiene Perceptrón simple, al ser útil únicamente para la resolución de problemas lineales. El Perceptrón multicapa es un modelo matemático que aproxima relaciones no lineales entre datos de entrada y salida. Es de fácil uso y uno de los más aplicados en la actualidad para resolver problemas no lineales. Sin embargo, tiene limitaciones, como el largo proceso de aprendizaje en el caso de problemas complejos con un gran número de variables o la problemática si se quiere realizar un análisis teórico de la red, a causa de la presencia de componentes no lineales.

Arquitectura

La arquitectura se basa en la agrupación de neuronas en diferentes capas. Cuenta con una capa de entrada, una capa de salida, y una o más capas ocultas, que son las encargadas del procesamiento no lineal de los patrones que entran.

La capa de entrada consta de neuronas que se encargan únicamente de recibir los patrones o señales del exterior y enviarlas a todas las neuronas de la capa siguiente. La última capa, la capa de salida, otorga al exterior una respuesta para cada patrón que entra en la red. Por otro lado, las neuronas de las capas intermedias o capas ocultas, llevan a cabo un procesamiento no lineal de los patrones que reciben.

En el Perceptrón multicapa, las conexiones de una capa a otra siempre están dirigidas hacia delante, es decir, las neuronas de una capa se conectan a las neuronas de la capa siguiente. Por este motivo se las conoce también como redes alimentadas hacia adelante o “feedforward”.

Habitualmente, las neuronas de una capa están conectadas a todas las neuronas de la capa siguiente, se dice entonces en estos casos que hay conectividad total.

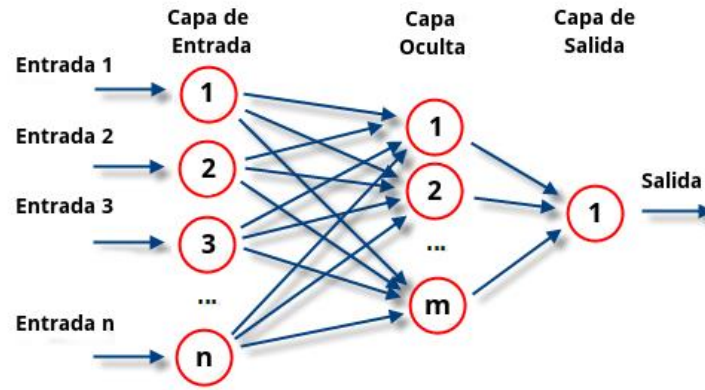


Figura 28 Arquitectura de una red neuronal tipo con una capa intermedia. Fuente: (Wikipedia, 2018)

A pesar de que la mayoría de las redes neuronales siguen el tipo de arquitectura mostrada en la Figura 19, se pueden encontrar también arquitecturas con otras características:

- Redes en las que todas o algunas de las neuronas de una capa están conectadas a neuronas de capas siguientes pero no de las inmediatamente siguientes.
- Redes en las que ciertas neuronas de ciertas capas no están conectadas a neuronas de la capa siguiente.

Propagación de los patrones de entrada

En las MLP cada neurona recibe información y genera una respuesta o activación que envía a las neuronas a las que está conectada. Sea una red de C capas y un número de neuronas n_c para la capa c (con $c = 1, 2, \dots, C$.) La representación de la matriz de pesos $W^c = (\omega_{ij}^c)$, donde ω_{ij}^c representa el peso de la conexión de la neurona i de la capa c , para las capas ocultas y de salida. Las activaciones, a_i^c se calcularían de la siguiente manera:

- Capa de entrada: Las activaciones en la capa de entrada son el vector de entrada a la red.

$$a_i^1 = x_i \text{ para } i = 1, 2, \dots, n_1$$

Donde $X = (x_1, x_2, \dots, x_{n_1})$ es el patrón de entrada.

- Capa oculta: Las neuronas de las capas ocultas realizan un procesamiento de la información que consisten en aplicar la función de activación f a la suma de los productos de las activaciones que recibe por sus pesos correspondientes:

$$a_i^c = f\left(\sum_{j=1}^{n_{c-1}} w_{ji}^{c-1} a_j^{c-1} + u_i^c\right) \text{ para } i = 1, 2, \dots, n_c \text{ y } c = 2, 3, \dots, C - 1$$

El término u_i^c hace referencia al umbral de la neurona. Este umbral se trata como una entrada más a la neurona que proviene de otra neurona siempre activa denominada *bias*. La función de esta neurona siempre activa es dar una entrada que juega el papel umbral a partir del cual la neurona a la que se conecta se activa. Es decir, podría entenderse como un peso más ($-w_i^{c-1}$) de tal forma que cuando el resultado de la suma de los productos de los pesos por las entradas recibidas es mayor que el umbral, la neurona se activa.

- Capa de salida: Las neuronas de la capa de salida funcionan igual que las de la capa oculta, es decir, procesas la información o activación recibida mediante la función de activación. En este caso, la salida corresponde con el vector de salida de la red $Y = (y_1, y_2, \dots, y_{n_c})$.

$$y_i = a_i^c = f\left(\sum_{j=1}^{n_{c-1}} w_{ji}^{c-1} a_j^{c-1} + u_i^c\right) \text{ para } i = 1, 2, \dots, n_c$$

Como función de activación, lo más frecuente es la función sigmoideal y la tangente hiperbólica:

Función sigmoideal: $f_{sigm}(x) = \frac{1}{1+e^{-x}}$

Función tangente hiperbólica: $f_{thip}(x) = \frac{1-e^{-x}}{1+e^{-x}}$

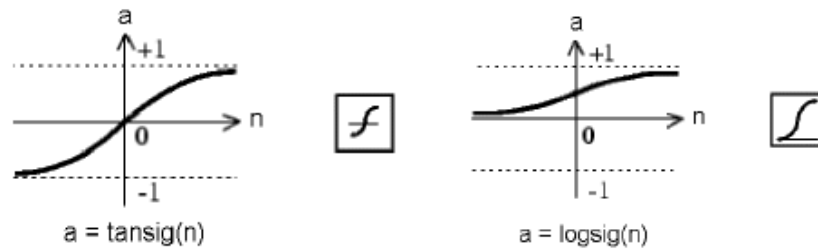


Figura 29 Funciones sigmoidal (izquierda) y tangente hiperbólica (derecha). Fuente: (Isasi Viñuela & Galvan, Redes de Neuronas Artificiales: Un Enfoque Práctico, 2004)

En el caso de la función sigmoidal el intervalo de valores es $[0, 1]$ mientras que en la tangente hiperbólica es $[-1, 1]$. Es decir, tienen un nivel de saturación máximo igual a 1 y difieren en el nivel de saturación mínimo, el cual es 0 para la función sigmoidal y -1 para la tangente hiperbólica. Estas funciones se relacionan a través de la ecuación $f_{thip}(x) = 2f_{sigm}(x) - 1$.

Consideraciones de diseño

Dependiendo del criterio del diseñador, en ocasiones las neuronas de salida tienen una función de activación diferente al resto de neuronas de las capas ocultas. Estas funciones en la salida, cuando se opta por que sean diferentes, suelen ser la función escalón o la función identidad.

Según lo comentado, en la utilización del Perceptrón multicapa, será necesario definir el número de neuronas de cada capa así como el número de capas ocultas de las que dispondrá la red. En el caso de las capas de entrada y salida, el número de neuronas viene fijado por el tipo de problema del que se trate, ya que el número de entradas y salidas de la red está determinado por las variables que definen el problema. No obstante, puede no conocerse con exactitud el número de variables relevantes para la red. Por ejemplo, en un problema con gran número de variables, algunas de ellas pueden no ser relevantes y su inclusión en la red podría suponer un aumento de la complejidad de la misma totalmente innecesario. Por este motivo es aconsejable realizar un análisis previo de las variables e identificar aquellas que tienen una significación en el caso de estudio.

En el caso de las capas ocultas la situación es diferente. Tanto el número de capas ocultas como el número de neuronas de estas capas quedan a elección

del diseñador. Lo más habitual es encontrar la arquitectura óptima mediante prueba y error de diferentes configuraciones

El número de neuronas ocultas puede influir en cómo se comporta la red pero, en el caso de las MLP, no se trata de una influencia muy significativa ya que, para un determinado problema, son muchas las arquitecturas capaces de resolverlo. La adición o eliminación de una neurona oculta no supone un cambio significativo en la capacidad de predicción de la red. No obstante, se encuentra en momento de investigación y desarrollo la línea de encontrar algún método que solucione la cuestión de definir el número de capas y neuronas ocultas más adecuado.

Algoritmo de retropropagación

Se ha comentado la arquitectura de las redes neuronales Perceptrón multicapa y las funciones de activación utilizables en las mismas pero, para hacer realmente posible la enseñanza de la red, es necesario un algoritmo que adapte y modifique los parámetros de la red llevando a cabo un aprendizaje.

El aprendizaje de la red es posible gracias a un algoritmo que, en el caso del Perceptrón multicapa, es supervisado. Esto quiere decir que se compara la salida de la red con la salida deseada, la cual es proporcionada por el supervisor. Para cada patrón de entrada, por tanto, es necesario tener un patrón de salida deseada, ya que el objetivo de la red es que la salida sea lo más próxima posible a la salida deseada. Se trata de un problema de minimización de la función de error. Dicha función de error calcula el promedio de los errores cometidos por la red para cada patrón. Habitualmente se define como:

$$E = \frac{1}{N} \sum_{n=1}^N e(n)$$

Donde N sería el número de muestras con las que entrenamos la red y $e(n)$ el error cometido para cada patrón n . Este error para cada patrón se

define como:

$$e(n) = \frac{1}{n_c} \sum_{i=1}^{n_c} (s_i(n) - y_i(n))^2$$

Donde $Y(n) = (y_1(n), \dots, y_{n_c}(n))$ y $S(n) = (s_1(n), \dots, s_{n_c}(n))$ son el vector que sale de la red y el vector de salida deseada respectivamente, para cada patrón n .

Al trabajar la red con funciones de activación no lineales, el problema de minimización del error es un problema no lineal y, por consiguiente, deben utilizarse métodos de optimización no lineales para su resolución. Estas técnicas usualmente se fundamentan en la modificación de parámetros siguiendo una dirección de búsqueda. En el caso de la MLP, esa dirección será la dirección negativa del gradiente de la función de error.

Como hemos visto, las salidas son el resultado de la aplicación de la función de activación a la suma de los productos de las activaciones de la capa anterior por su peso correspondiente. Esta dependencia con los pesos lleva al resultado de que, para encontrar el mínimo de la función de error, se deben modificar los pesos $w(n)$ correspondientes a cada neurona.

Generalmente, el método utilizado para la minimización del error está basado en la minimización de cada error $e(n)$ aplicando el método del gradiente estocástico. Esto se traduce en la siguiente ley de aprendizaje:

$$w(n) = w(n-1) - \alpha \frac{\partial e(n)}{\partial w}$$

Donde α es la razón o tasa de aprendizaje, de la cual se hablará más adelante.

Este método del gradiente, aplicado a las MLP, se denomina de algoritmo de retropropagación porque el error de la salida de la red es propagado hacia atrás, transformándolo en un error en cada una de las neuronas de las capas ocultas de la red.

Obtención de la regla delta generalizada

Se va a desarrollar seguidamente la obtención del algoritmo de retropropagación, también conocido como regla delta generalizada. Este algoritmo es diferente para el caso de la adaptación de los pesos de la penúltima capa a la capa de salida respecto a la adaptación de los pesos del resto de la red. Se detallan por separado los procedimientos para los dos casos.

Si consideramos w_{ji}^{C-1} el peso de la conexión de la neurona j de la penúltima capa ($C - 1$) a la neurona i de la capa de salida, según lo desarrollado anteriormente, dicho error se modifica según:

$$w_{ji}^{C-1}(n) = w_{ji}^{C-1}(n-1) - \alpha \frac{\partial e(n)}{\partial w_{ji}^{C-1}}$$

La derivada del error respecto al peso en ese punto, teniendo en cuenta la expresión del error, y que las salidas deseadas $s_i(n)$ son constantes, resulta que:

$$\frac{\partial e(n)}{\partial w_{ji}^{C-1}} = -(s_i(n) - y_i(n)) \frac{\partial y_i(n)}{\partial w_{ji}^{C-1}}$$

Para el cálculo, a continuación, de la derivada de la salida $y_i(n)$ respecto al peso w_{ji}^{C-1} , se tiene en cuenta que la salida es el resultado de la aplicación de la función de activación. Derivando aplicando la regla de la cadena se obtiene que:

$$\frac{\partial y_i(n)}{\partial w_{ji}^{C-1}} = f' \left(\sum_{j=1}^{n_C-1} w_{ji}^{C-1} a_j^{C-1} + u_i^C \right) a_j^{C-1}(n)$$

En este punto, se define el término δ correspondiente a la neurona i de la capa C (de salida) y asociado al patrón n :

$$\delta_i^C(n) = -(s_i(n) - y_i(n)) f' \left(\sum_{j=1}^{n_C-1} w_{ji}^{C-1} a_j^{C-1} + u_i^C \right)$$

Reemplazando los valores de la derivada de la salida y el valor de δ_i^C

calculado, resulta:

$$\frac{\partial e(n)}{\partial w_{ji}^{C-1}} = \delta_i^C(n) a_j^{C-1}(n)$$

Por último, se sustituye por su desarrollo la expresión de la derivada del error respecto a w_{ji}^{C-1} y se obtiene la ley para modificar el peso:

$$w_{ji}^{C-1}(n) = w_{ji}^{C-1}(n-1) + \alpha \delta_i^C(n) a_j^{C-1} \text{ para } j = 1, 2, \dots, n_{C-1} \text{ } i = 1, 2, \dots, n_C$$

De la observación de esta expresión se extrae que la modificación del peso de una conexión se considera la activación de la neurona de la que proviene la conexión y el término δ de la neurona a la que llega la conexión.

Se analiza de igual manera la ley de adaptación de pesos para las conexiones de las capas intermedias de la red. Se considera la conexión de la capa $C-2$ a la capa $C-1$ para, posteriormente, proceder a su generalización. La ley de actualización de este peso w_{kj}^{C-2} es:

$$w_{kj}^{C-2}(n) = w_{kj}^{C-2}(n-1) - \alpha \frac{\partial e(n)}{\partial w_{kj}^{C-2}}$$

En esta ocasión, el peso w_{kj}^{C-2} tiene influencia en todas las neuronas de salida de la red, entonces la derivada del error respecto a este peso está determinado por la suma de las derivadas respecto a todas las neuronas de salida:

$$\frac{\partial e(n)}{\partial w_{kj}^{C-2}} = - \sum_{i=1}^{n_C} (s_i(n) - y_i(n)) \frac{\partial y_i(n)}{\partial w_{kj}^{C-2}}$$

Desarrollando la expresión de la derivada de la salida respecto a dicho peso y, teniendo en cuenta que el peso influye en la activación de la neurona j de la capa siguiente se obtiene:

$$\frac{\partial y_i(n)}{\partial w_{kj}^{C-2}} = f' \left(\sum_{j=1}^{n_{C-1}} w_{ji}^{C-1} a_j^{C-1} + u_i^C \right) w_{ji}^{C-1} \frac{\partial a_j^{C-1}}{\partial w_{kj}^{C-2}}$$

Sustituyendo en la ecuación anterior y de acuerdo a la definición de δ

resulta:

$$\frac{\partial e(n)}{\partial w_{kj}^{c-2}} = \sum_{i=1}^{n_c} \delta_i^c(n) w_{ji}^{c-1} \frac{\partial a_j^{c-1}}{\partial w_{kj}^{c-2}}$$

A continuación realizamos la derivada de la activación de la neurona j respecto al peso. Se obtiene:

$$\frac{\partial a_j^{c-1}}{\partial w_{kj}^{c-2}} = f' \left(\sum_{k=1}^{n_{c-2}} w_{kj}^{c-2} a_k^{c-2} + u_j^{c-1} \right) a_k^{c-2}(n)$$

Se define, para las neuronas de la penúltima capa $\delta_j^{c-1}(n)$:

$$\delta_j^{c-1}(n) = f' \left(\sum_{k=1}^{n_{c-2}} w_{kj}^{c-2} a_k^{c-2} + u_j^{c-1} \right) \sum_{i=1}^{n_{c-2}} \delta_i^c(n) w_{ji}^{c-1}$$

Por tanto, sustituyendo con $\delta_j^{c-1}(n)$ resulta:

$$\frac{\partial e(n)}{\partial w_{kj}^{c-2}} = \delta_j^{c-1}(n) a_k^{c-2}(n)$$

Y como resultado, la ley de aprendizaje para este peso se define como:

$$w_{kj}^{c-2}(n) = w_{kj}^{c-2}(n-1) + \alpha \delta_j^{c-1}(n) a_k^{c-2}(n)$$

$$\text{para } k = 1, 2, \dots, n_{c-2} \text{ y } j = 1, 2, \dots, n_{c-1}$$

Se puede observar que, también en este caso, es suficiente con tener en cuenta la activación de la neurona de la que parte la conexión y el término δ de la neurona a la que llega la conexión. Una vez estudiada la ley de aprendizaje para la capa $C-2$, es posible su generalización para el resto de capas c tales que $c = 1, 2, \dots, C-2$, quedando:

$$w_{kj}^c(n) = w_{kj}^c(n-1) + \alpha \delta_j^{c+1}(n) a_k^c(n)$$

$$\text{para } k = 1, 2, \dots, n_c, j = 1, 2, \dots, n_{c+1} \text{ y } c = 1, 2, \dots, C-2$$

Para la conexión de los umbrales se aplica la misma ley de aprendizaje, pero en este caso la activación es constante.

$$u_j^{c+1}(n) = u_j^c + 1(n-1) + \alpha \delta_j^{c+1}(n)$$

La obtención de los valores de δ , como se ha mostrado, pasa por el cálculo de la derivada de la función de activación. Puesto que la función sigmoideal es ampliamente utilizada como función de activación, se detalla la obtención de su derivada de la siguiente manera:

$$f'(x) = \frac{1}{(1 + e^{-x})^2} (-e^{-x}) = \frac{1}{1 + e^{-x}} \frac{e^{-x}}{1 + e^{-x}}$$

Razón de aprendizaje. Inclusión del momento en la ley de aprendizaje

Se ha explicado la obtención de la regla delta, o algoritmo de retropropagación. En la expresión de este algoritmo se ha señalado la aparición del término α : la razón o tasa de aprendizaje. Este término multiplica a la derivada del error como se mostró anteriormente:

$$w(n) = w(n - 1) - \alpha \frac{\partial e(n)}{\partial w}$$

Se procede ahora a la explicación de este término y a la inclusión de un nuevo parámetro en la ley de aprendizaje: el momento.

La tasa de aprendizaje es un número que multiplica a la derivada del error respecto del peso, es decir, es un parámetro que marca la magnitud del desplazamiento del peso respecto al gradiente del error. En otras palabras, una tasa de aprendizaje mayor, implica una alta velocidad de convergencia del algoritmo. No obstante, elegir una tasa de aprendizaje alta puede ocasionar que el algoritmo se salte el mínimo de la función de error o que oscile entorno a él. Por otro lado, una tasa de aprendizaje baja, aunque disminuye la probabilidad de que esto ocurra y suele dar resultados más acertados, también provoca mayor lentitud en la convergencia del algoritmo.

Un método para paliar la posible inestabilidad del algoritmo de aprendizaje es la inclusión de un término que modifica la ley de aprendizaje. Este término es el momento.

$$w(n) = w(n - 1) - \alpha \frac{\partial e(n)}{\partial w} + \eta \Delta w(n - 1)$$

Donde $\Delta w(n - 1) = w(n - 1) - w(n - 2)$ y η es un número positivo que

define el peso que se le da a este incremento de w .

Proceso de aprendizaje del perceptrón multicapa

Una vez vista la arquitectura de la red, analizada la regla delta generalizada y la ley de aprendizaje de la misma, se procede a explicar el proceso de aprendizaje durante el entrenamiento de la red. Se parte de unos patrones iniciales entre los que se puede diferenciar un vector de muestras de entrada $\{X(n) = (x_1(n), x_2(n), \dots, x_{n1}(n))\}$ y un vector de salidas deseadas $\{S(n) = (s_1(n), s_2(n), \dots, s_{nc}(n))\}$. Habitualmente estos patrones se normalizan en una escala $[0,1]$ o $[-1,1]$ según la función de activación que se vaya a emplear. En el entrenamiento de la red se siguen los siguientes pasos:

- 1) Mediante valores aleatorios se inicializan los umbrales y pesos de la red.
- 2) Se introduce a la red un patrón de entrada y se propaga hacia la salida, produciéndose una respuesta para ese patrón de entrada.
- 3) Se compara la respuesta obtenida con la salida deseada. En este paso evaluamos el error cuadrático cometido para ese patrón.
- 4) Se procede a la aplicación de la regla delta generalizada con el objetivo de modificar los pesos y umbrales de la red. Para ello, se calculan los valores δ para las neuronas de salida. A continuación se calculan para el resto de neuronas de la red retropropagándose hacia detrás. Finalmente, se aplica la regla modificando así los pesos y umbrales de la red.
- 5) Se repiten los pasos realizados para el resto de patrones de la muestra. Una vez se han introducido todos los patrones de la muestra, se ha completado un ciclo, también llamado *epoch*.
- 6) Completado el ciclo, se evalúa el error total cometido por la red. Este error es el error de entrenamiento, ya que se ha calculado con los patrones que se utilizan para el entrenamiento de la red.
- 7) Se repite el entrenamiento en sucesivos ciclos hasta que se alcanza un mínimo del error de entrenamiento.

En ocasiones, se puede no encontrar un mínimo global si existen mínimo locales. Añadir el parámetro del momento en la ley de aprendizaje y utilizar distintos valores iniciales de los pesos puede aumentar la probabilidad de encontrar un mínimo global.

Las redes neuronales deben reentrenarse periódicamente para su adaptación al mercado. Si no se reentrena, ésta puede perder precisión en su funcionamiento. Por lo general, la red mejorara cuanto más frecuentemente se entrene.

Entrenar sucesivamente la red con los mismos patrones puede llevar a un problema de sobreentrenamiento (*overfitting*). Esto implica la llegada a un punto en el que el error cometido para esos patrones de entrenamiento disminuye pero a costa de perjudicar la capacidad de generalización de la red. Existen diferentes métodos para parar el entrenamiento de la red y evitar así el sobreentrenamiento de la misma. Uno de estos métodos es el conocido como *early stopping*.

Early stopping

El método de *early stopping* consiste en la extracción previa al entrenamiento de un número de muestras de entrada con sus respectivas salidas deseadas. Estas muestras no se utilizan en el entrenamiento de la red. Tras haber completado un ciclo de entrenamiento, se introducen en la red las muestras separadas para su validación del error. A continuación se continúa con el entrenamiento de la red, volviendo a evaluar el error cometido sobre las muestras de validación tras la finalización de un ciclo. El entrenamiento se detendrá cuando el error cometido sobre las muestras de validación aumente. Esta parada es la denominada *early stopping* (Isasi Viñuela & Galvan, Redes de Neuronas Artificiales: Un Enfoque Práctico, 2004).

Además del procedimiento de *early stopping*, hay otros métodos para evitar el sobreentrenamiento de la red. Uno de ellos es fijar un número de ciclos o pasos máximo que se realizarán durante el entrenamiento.

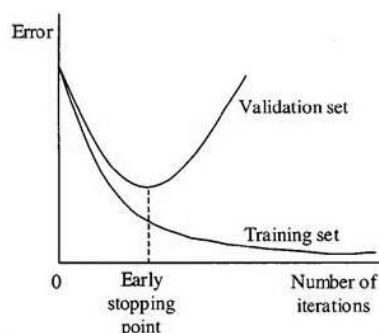


Figura 30 Representación del criterio de parada "early stopping". Fuente: (Isasi Viñuela & Galvan, Redes de Neuronas Artificiales: Un Enfoque Práctico, 2004)

Alcanzado este número de pasos, el proceso se detendrá y finalizará el entrenamiento. Otra opción para detener la red es determinar un valor (*threshold*) para la derivada parcial de la función de error de forma que, si se alcanza ese valor, el entrenamiento de la red se detiene (Isasi Viñuela & Galvan, Redes de Neuronas Artificiales: Un Enfoque Práctico, 2004).

Se va a utilizar en este estudio el paquete “neuralnet” de Rstudio. Se cuenta con una base de datos en Excel de 3265 datos de 8 variables (“Vacío”, “Lectura1”, “Lectura2”, “Alfa”, “Beta”, “Gamma”, “Theta”, “Delta”) donde se encuentran todos los registros realizados en los 3 escenarios. Estos datos se normalizan en la escala [0,1] debido a que se va a utilizar la función de activación sigmoideal. Los patrones de entradas son los vectores cuyas componentes son los datos registrados para cada onda cerebral en cada registro. La salida deseada para cada patrón de entrada es un vector de 3 componentes con valor de 1 en la 1ª, 2ª o 3ª componente en función de si la salida es el escenario vacío, la lectura 1 o la lectura 2. Se tiene, por tanto, una capa de entrada de 5 neuronas y una capa de salida de 3 neuronas.

El algoritmo utilizado es el de retropropagación, con una tasa de aprendizaje por defecto que se modifica según el menor gradiente del error. Como criterio de parada se limita el número de pasos a e^6 y se incluye por defecto, además, un *threshold* de 0.01.

De los 3265 patrones de los que se dispone, se ha utilizado una muestra de 2612 datos para el entrenamiento de la red y 653 para el testeo posterior de la misma. Estas muestras se guardan en dos vectores de datos con el

programa R Studio. Se hacen 2 redes neuronales, una con 1 capa oculta de 4 neuronas y otra que consta de 2 capas ocultas con 4 y 2 neuronas respectivamente.

Modelo

En este epígrafe se describe el trabajo realizado para obtener un modelo que permita estimar las fases cognitivas de una persona a partir de la actividad cerebral medida mediante un EEG. Para ello, se expone primeramente la descripción de la red MLP que se ha programado en lenguaje R, para después exponer las diferentes características de red y resultados del entrenamiento realizado. Se diseñan dos redes MLP que difieren en el número de capas y neuronas ocultas a fin de obtener una idea del tipo de solución que da mejores resultados. Al final del capítulo se exponen las conclusiones y los detalles de la capacidad de clasificación de ambas redes.

MLP - Una capa oculta

En una primera fase de estudio se utiliza una red neuronal Perceptrón multicapa con una única capa oculta con cuatro neuronas. Se trata de una arquitectura sencilla con un número de neuronas ocultas intermedio entre el número de neuronas en la capa de entrada y en la capa de salida.

Esta primera configuración que se prueba consta, por tanto, de 3 capas: la capa de entrada con 5 neuronas, una capa oculta de 4 neuronas y una capa de salida de 3 neuronas. Todas las neuronas de una capa están conectadas a todas las neuronas de la capa siguiente, es decir, la conexión es total.

Como primer paso se prepara la base de datos con la que se va a trabajar. Disponemos de un fichero con los valores registrados con la diadema Mindwave para las 5 ondas cerebrales (alfa, delta, theta, beta y gamma). Este fichero lo normalizamos con R Studio a un intervalo de 0 a 1. Este paso es aconsejable para evitar errores en la comparación de entradas con diferentes órdenes de magnitud. La instrucción principal del paquete

“neuralnet” que permite el diseño y entrenamiento de la red neuronal es la función `neuralnet()`. Esta función la implementamos como:

```
neuralModel<-neuralnet(Vacio+Lectura1+Lectura2~Alpha+Delta+Gamma+Theta+Beta,hidden=c(4),stepmax=1e6,linear.output=FALSE,data=train)
```

Según los parámetros especificados y por los procesos que lleva a cabo la función si no se especifica lo contrario como parámetro, se han realizado los siguientes puntos del diseño:

- Se define la capa de salida con las 3 salidas de los escenarios, y la capa de entrada con las 5 ondas cerebrales.
- Se inicializan de manera aleatoria los pesos y umbrales iniciales de la red, es decir, los valores umbrales a partir de los cuales una neurona se activa, así como los pesos que van a multiplicar a las activaciones, comienzan siendo números aleatorios.
- Se implementa una capa oculta de 4 neuronas.
- Se define la función de activación sigmoideal. El motivo es que esta función tiene un intervalo de 0 a 1, y la normalización de los datos se ha realizado en este intervalo, por tanto se aconseja el uso de esta función. Esta función, al rechazar la salida lineal en la función “neuralnet”, es la función de activación por defecto.
- Se indican como criterios de parada un *threshold* de 0.01 (por defecto) y un n° de pasos máximo de e^6 . Se llevará a cabo la parada que primero se cumpla de las dos.
- Se introduce en la red el vector de datos “train”, en el que se encuentran los patrones de entrenamiento. El paquete neuralnet nos permite la visualización de la red en un diagrama. En la arquitectura que dibuja el programa, las líneas negras muestran las conexiones entre cada capa y los pesos de cada conexión, mientras que las líneas azules muestran el valor del umbral agregado en cada conexión. Como se ha mostrado al principio del capítulo, estos pesos son los que se multiplican por las activaciones que cada neurona recibe.

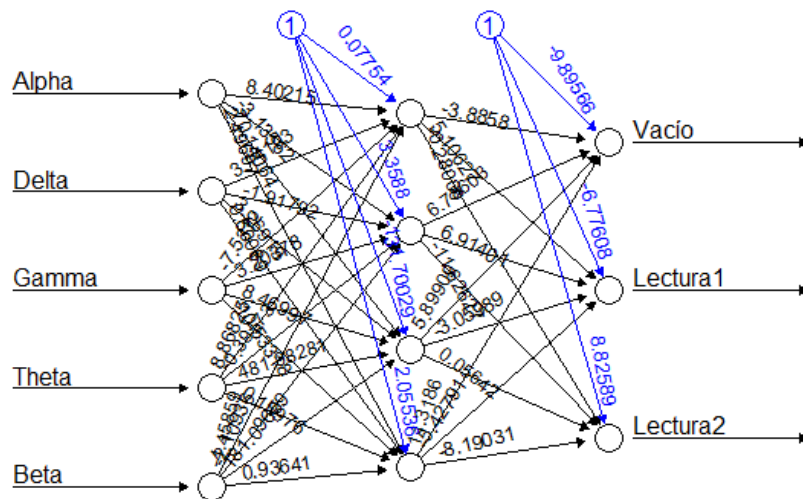


Figura 31 Configuración red neuronal con 1 capa oculta de 4 neuronas. Fuente: Elaboración propia

Por otro lado, los umbrales son los valores que el sumatorio de entradas a la neurona, por el peso asociado a la misma, debe cumplir para que la neurona se active, emitiendo una salida.

Una vez finalizado diseñado la red, ejecutando la función “neuralnet” comienza el entrenamiento de la misma. En este paso el programa introduce los patrones que se han reservado en un vector aparte para el entrenamiento. Este es el vector que hemos denominado “train”.

Terminado el entrenamiento, se procede a testear la red con los datos reservados para la prueba de la MLP. Para ello se pasan a la red los patrones de entrada que se encuentran en el vector preparado para el testeo. Para realizar este paso, se ejecuta la función compute().

```
clasificaciones<-compute(neuralModel,test[,3:7])
```

En esta función se ha especificado la red que queremos testear en primer lugar. En segundo lugar se pasa como parámetro un vector que se ha denominado “test”. En este vector se encuentran los patrones de prueba que se han preparado. Es importante señalar que este vector contiene patrones que no se han utilizado durante el entrenamiento. Entre corchetes se especifica las columnas en las que se encuentran los valores de las 5 ondas cerebrales, que serán las entradas a la red.

Una vez probada la red, se recogen las clasificaciones que la red da como resultado, las cuales se encuentran en la lista “clasificaciones”. Algunos de

los resultados, a modo de ejemplo, se muestran en la siguiente tabla:

Nº de patrón	Resultado		
	Vacío	Lectura1	Lectura2
1	0,824690647	0,156346656	0,0727852
12	0,843613892	0,245302192	0,028244968
14	0,723073786	0,325027974	0,054270636
15	0,864964768	0,095763112	0,074197845
26	0,971964616	0,306240737	0,003069616

Tabla 2 Muestra de 5 resultados de la red de 1 capa oculta de 4 neuronas para patrones de entrada pertenecientes al escenario "Vacío"

Los resultados de la red son 3 valores cuya suma es uno, en los que uno de los escenarios predomina sobre el resto. Para validar la red, se comparan estos resultados con la clasificación real que nos dan los patrones de salida deseada. Este paso se realiza de manera sencilla importando los datos a Excel y realizando en este programa la comparación y graficación de los resultados.

MLP - Dos capas ocultas

En la siguiente fase de estudio se decidió comparar los resultados anteriores con los de una red MLP de dos capas ocultas, con cuatro y dos neuronas, para ver las diferencias en los resultados que se obtienen. Esta comparación podría darnos una idea del efecto que supone la adición de capas a la red. Como se señaló anteriormente, en cuestión de diseño, no se encuentra actualmente un método diseñado que permita determinar previamente a la elección de una red neuronal qué arquitectura es la más adecuada para la resolución de un problema.

La segunda configuración que se prueba, como contraposición a la primera, consiste en la adición de una capa oculta más de 2 neuronas. Por consiguiente, esta red consta de 4 capas: la capa de entrada con 5 neuronas, una capa oculta de 4 neuronas, una segunda capa oculta de 2 neuronas y una capa de salida de 3 neuronas. Todas las neuronas de una capa están conectadas a todas las neuronas de la capa siguiente, la conexión es total.

Procediendo de igual manera que con la red anterior, con la función `neuralnet()`, se inicializan de manera aleatoria los pesos y umbrales iniciales de la red, se define la función de activación sigmoideal y los criterios de parada con un *threshold* de 0.01 y un n° de pasos máximo de e^6 . Igualmente se determinan las capas de la red.

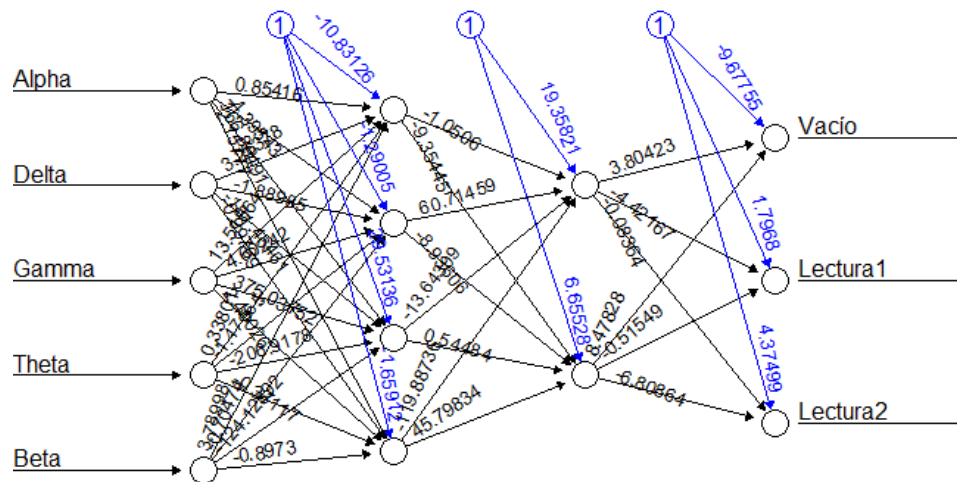


Figura 32 Configuración red neuronal con 2 capas ocultas de 4 y 2 neuronas respectivamente. Fuente: Elaboración propia

Tras el diseño de la red neuronal, mediante la función `compute()` explicada en el caso anterior, se procede al entrenamiento de la red. De igual manera, se emplea un vector con patrones de entrenamiento. Una vez finalizado el entrenamiento de la red, se prueba su capacidad de clasificación con los datos reservados para la prueba de la misma, es decir, con el vector de patrones de testeo. Para ello se pasan a la red los patrones de entrada de esta muestra y se recogen las clasificaciones que la red da como resultado. Los resultados de esta red son del mismo tipo que los obtenidos con la red de una capa. Pertenecen al escenario vacío.

N° de patrón	Resultado		
	Vacío	Lectura1	Lectura2
1	0,920343546	0,048246585	0,076142919
12	0,930984601	0,041480623	0,074824265
14	0,930944803	0,041482008	0,074858835
15	0,927897489	0,041594423	0,077484267
26	0,93117606	0,041474176	0,074657621

Figura 33 Muestra de 5 resultados de la red de 2 capas ocultas para patrones del escenario "Vacío"

Para su validación, se comparan estos resultados con la clasificación real que nos dan los patrones de salida deseada.

Análisis del Modelo

Obtenidos los resultados para todas las muestras de testeo se procede a su comparación con los valores reales o salidas deseadas, es decir, se contrasta, para cada patrón utilizado en la prueba de la red, el escenario real al que pertenece el patrón de valores con el escenario en el que la red neuronal ha clasificado el patrón.

Se va a comparar los resultados de cada red por separado y después la capacidad de clasificación de una red respecto a la otra.

Con la red de una capa oculta, se comparan las salidas otorgadas por la red con las salidas que habrían sido deseadas que saliesen, o sea con los valores reales, y se obtiene un porcentaje de acierto de 74,3%.

		Salida obtenida		
		Vacío	Lectura1	Lectura2
Salida deseada	Vacío	163	26	0
	Lectura1	103	68	0
	Lectura2	38	1	254

Tabla 3 Resultados obtenidos con la red de 1 capa oculta de 4 neuronas comparados con la salida deseada

El ratio de acierto para cada escenario es de 86,2% para el escenario “Vacío”, 39,8% para “Lectura1” y 86,7% para “Lectura2”.

Se observa una clara confusión en la clasificación del escenario de “Lectura1”, la lectura científica. Los patrones de esta lectura se clasifican en su mayoría como si perteneciesen al escenario vacío.

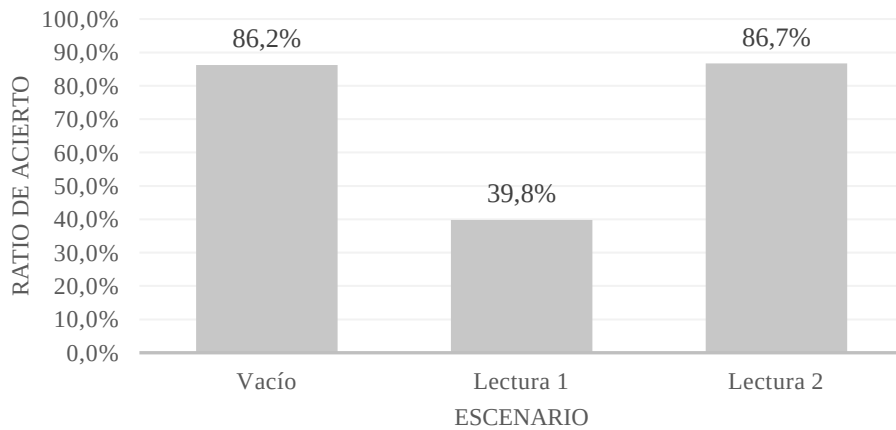


Figura 34 Porcentaje de acierto para cada escenario con la red neuronal de 1 capa oculta de 4 neuronas.
Fuente: Elaboración propia

Una vez analizados los resultados con esta configuración de red, se procede a comprobar si con la inclusión de una capa oculta más la capacidad de clasificación de los patrones aumenta.

Seleccionando ahora la red de dos capas ocultas, y una vez obtenidos los resultados para todas las muestras de testeo, se procede a su comparación con los valores reales o salidas deseadas. De esta comparación se obtiene un porcentaje de acierto de 68,0%.

		Salida obtenida		
		Vacío	Lectura1	Lectura2
Salida deseada	Vacío	187	2	0
	Lectura1	170	1	0
	Lectura1	37	0	256

Tabla 4 Resultados obtenidos con la red de 2 capas ocultas de 4 y 2 neuronas comparados con la salida deseada

El ratio de acierto para cada escenario es de 98,9% para el escenario “Vacío”, 0,6% para “Lectura1” y 87,4% para “Lectura2”. Vuelve a llamar la atención, aunque esta vez de forma más llamativa, la clasificación prácticamente total de los patrones del escenario “Lectura1” como si fuesen patrones del escenario “Vacío”.

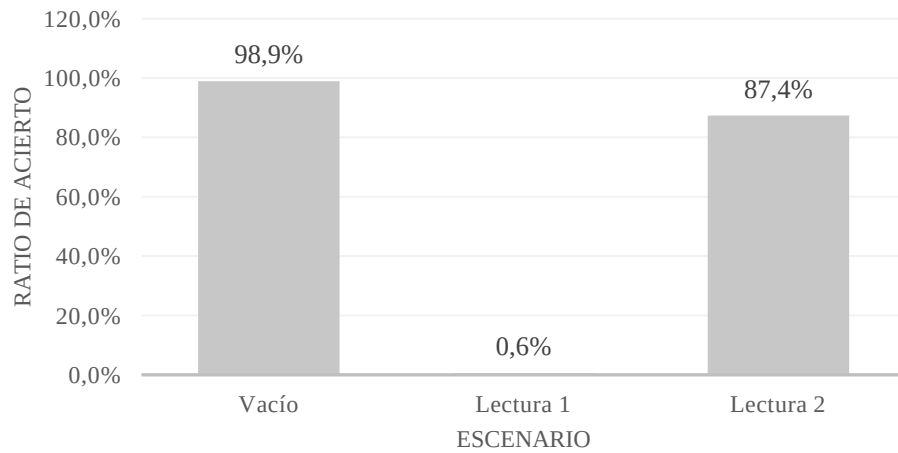


Figura 35 Porcentaje de acierto para cada escenario con la red neuronal de 2 capas ocultas de 4 y 2 neuronas. Fuente: Elaboración propia

A la vista de los resultados de las dos redes implementadas, podemos concluir que la red de una única capa oculta ofrece mejores resultados que la red con dos capas ocultas, siempre y cuando se quiera clasificar los tres tipos de patrones. Sin embargo, quizá si diseñáramos un problema de clasificación con dos escenarios marcadamente diferentes, como son el “Vacío”, en el que la persona no lee, y un escenario como el de “Lectura2”, en el cual la persona lee un texto a priori complicado, sería aconsejable la utilización de una red neuronal con 2 capas. Esta red tiene una capacidad de clasificación muy alta para estos dos escenarios, sin embargo, no es capaz de reconocer un escenario de lectura científica.

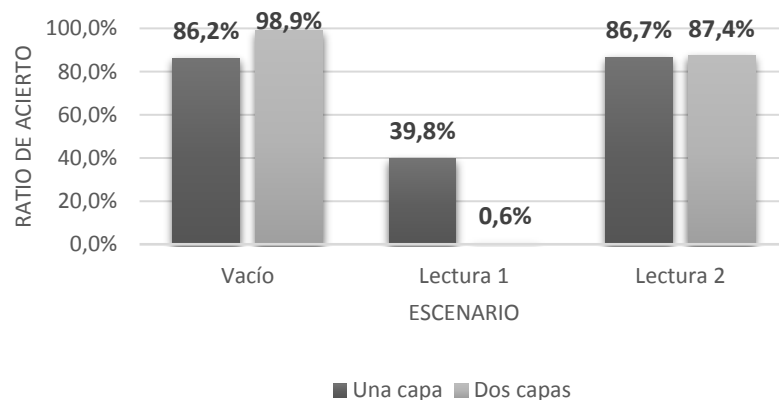


Figura 36 Comparación de la capacidad de clasificación de las dos redes neuronales implementadas. Fuente: Elaboración propia

Capítulo 6

DISCUSIÓN Y TRABAJO FUTURO

Se han analizado tres escenarios diferentes en los que dos de ellos involucran la actividad de lectura. Esta actividad pone en marcha diferentes procesos cognitivos que dan como resultado una diferencia en la actividad cerebral traducida en diferentes valores de las ondas cerebrales medidas.

Del análisis estadístico realizado hemos comprobado que, efectivamente, el escenario es un factor influyente en los valores de las ondas cerebrales. Que una persona esté leyendo, o no, supone una diferencia notable en su actividad cerebral y esta diferencia es medible a partir de sus ondas cerebrales.

Los valores de las ondas cerebrales registradas aumentan desde el escenario de no lectura, a los escenarios de lectura, siendo mayor en el escenario del libro sobre emociones. Esta última lectura es la que más esfuerzo ha supuesto para el lector, por lo que se puede concluir que a mayor exigencia de atención del texto, mayor actividad de las ondas cerebrales se registra con la diadema Mindwave.

Por otro lado, tras el análisis de la respuesta dada a las preguntas sobre el texto leído, se observa que este aumento de la actividad de las ondas cerebrales se produce en las ondas delta, theta y alfa cuando no se sabe la respuesta a la pregunta. Es decir, el mayor esfuerzo que supone tratar de recordar algún fragmento del texto, provoca un registro de valores mayores de estas ondas cerebrales. Sin embargo, en las ondas beta y gamma se produce un descenso de su nivel de actividad cuando se desconoce la respuesta a la pregunta. Este resultado nos muestra una diferencia de

comportamiento entre las ondas de más baja frecuencia (delta, theta y alfa) y las de más alta frecuencia (beta y gamma).

Desde un punto de vista técnico, se ha comprobado la viabilidad del dispositivo Mindwave de Neurosky para la realización de proyectos de este estilo. Resulta posible la identificación de diferentes estados cognitivos con el uso de esta diadema. Bien es cierto que para un análisis más preciso podría ser aconsejable la utilización de un dispositivo con mayor número de electrodos. Eso sí, el aumento de electrodos acarrearía en la mayoría de los casos un aumento del coste del proyecto. En la búsqueda de un equilibrio de coste y precisión, la diadema Mindwave resulta ser un dispositivo óptimo.

Se han realizado dos redes neuronales entrenadas con una muestra de los patrones registrados para producir la clasificación de futuros patrones que se introduzcan en la red. De las dos redes implementadas, se observa una clara diferencia en la clasificación del escenario de lectura científica. La red de una sola capa oculta clasifica 66 veces mejor el escenario científico que la red con dos capas ocultas, siendo esta última prácticamente incapaz de diferenciar el escenario científico del escenario vacío. La adición de una capa, sin embargo, mejora la precisión en la clasificación de los escenarios vacío y de lectura sobre emociones. En cualquier caso, se ha conseguido el objetivo de hacer una clasificación de estados cognitivos con redes neuronales.

En otros estudios relacionados se asocia a un estado de atención el incremento de actividad de las ondas beta y gamma mientras que la onda alfa se asocia a estados de falta de concentración (Sanei & Chambers, 2013). Los resultados del proyecto realizado muestran un incremento de actividad de todas las ondas cerebrales al aumentar la dificultad del texto y no únicamente de beta y gamma. No se ha observado una diferencia de comportamiento notable entre las diferentes ondas cerebrales, por lo tanto no se confirma un incremento de nivel proporcional en todas las ondas con la dificultad de las lecturas.

En cuanto al nivel de actividad de las ondas cerebrales durante las respuestas a las preguntas, se obtiene que las ondas alfa, theta y delta

disminuyen cuando se conoce la respuesta a la pregunta, mientras que beta y gamma aumentan. Este resultado parece contrario a la relación de la atención con el incremento de gamma y beta pero sería necesaria una investigación más detallada de los procesos desencadenados en la respuesta a las preguntas ya que podría estar influyendo, entre otros, el estado emocional.

Entre las posibles futuras investigaciones a realizar, sería interesante investigar la lectura sin leer en voz alta. En este proyecto se ha leído en voz alta, lo cual podría suponer una participación más activa de la zona del lenguaje y, por consiguiente, un estado mental diferente a la lectura sin hablar. Podría investigarse si este factor marca una diferencia notable.

De igual manera, para una clasificación más robusta, es aconsejable la utilización de un mayor número de sujetos. Especialmente si la investigación está orientada a aplicaciones de uso para un gran volumen de usuarios, lo cual se cumplirá en numerosas ocasiones dado que son muchas las posibles aplicaciones de la capacidad de clasificación de estados cognitivos mediante un EEG.

Entre ellas podríamos destacar el campo de la conducción, en el que conocer el nivel de atención del conductor puede suponer un avance a la seguridad vial, por ejemplo, sugiriendo al conductor una parada si se está quedando dormido. Además, como se comenta al principio del trabajo se busca estudiar procesos en salud mental, observando los estados que se dan en la mente del usuario durante el proceso terapéutico, como complemento a la terapia. En las enfermedades, tales como la ELA se presenta como una gran ayuda para personas que no pueden hablar, pudiendo facilitar así la comunicación con su entorno, véase por ejemplo mostrarse hambriento, con sueño, con ganas de ver una película o poder pedir ayuda a través de una tablet que tenga identificados dichos estados. En esta línea ya se realizan estudios orientados a la ayuda a la comunicación de pacientes con ELA (Birbaumer, y otros, 2000).

Otro posible campo de aplicación es la educación, ya que en el proceso de aprendizaje es necesario un determinado nivel de concentración por parte

del alumno. Si un determinado número de niños pierde la atención durante una clase, quizás sea momento para un descanso, o para la búsqueda de diversas metodologías alternativas de enseñanza.

En definitiva, la capacidad de reconocimiento del estado mental de una persona tiene aplicación en todos aquellos campos o trabajos en los que se requieran unos intervalos de actividad mental determinados. Esta meta pasa por un camino de investigación y trabajo en la línea de mejora de los EEG y las redes neuronales.

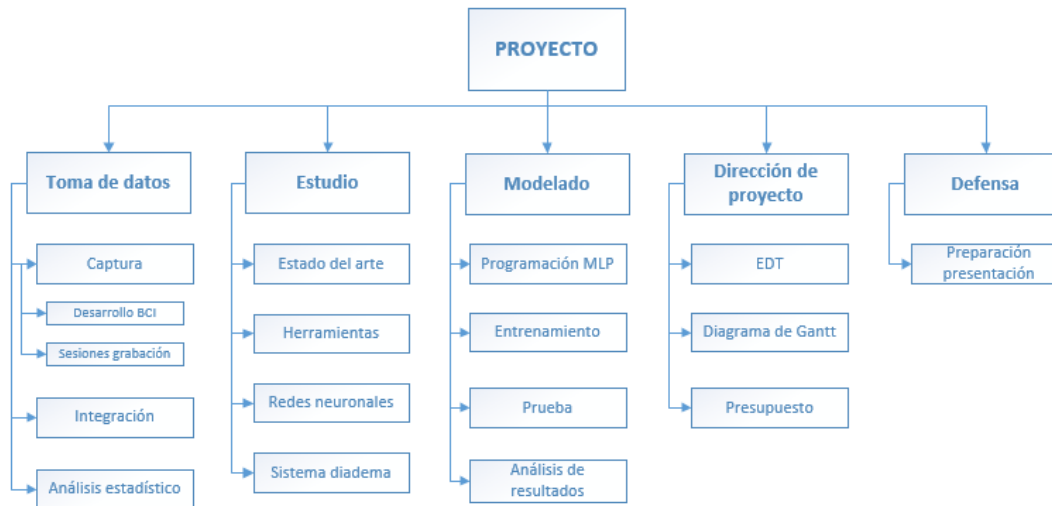
Capítulo 7

PLANIFICACIÓN Y PROGRAMACIÓN

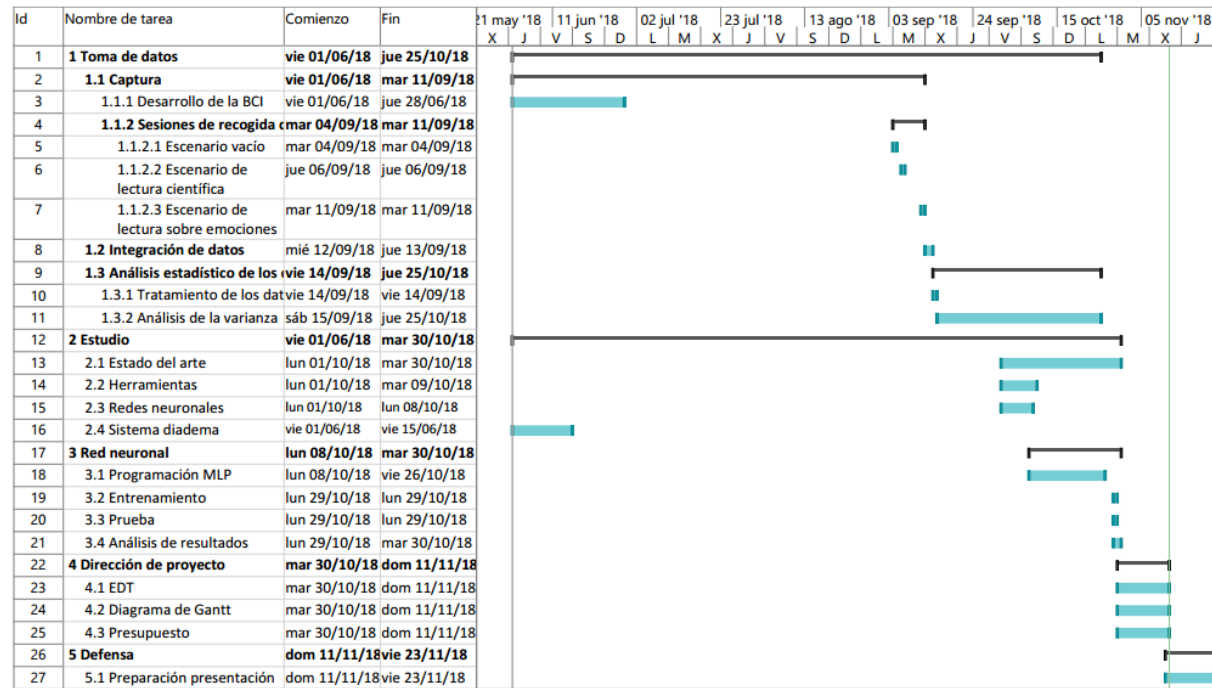
La metodología PMI es el método propuesto por el Project Management Institute donde se establecen los estándares que orientan la gestión de proyectos. Entre los documentos que sirven como base para la documentación de los proyectos en general, se incluyen:

- La EDT (*Estructura de Descomposición de Trabajo*), que es una descomposición del trabajo que será ejecutado.
- El cronograma, que determina las fechas planificadas para cada una de las actividades del proyecto. Se acostumbra a utilizar el diagrama de Gantt por la visibilidad que ofrece.
- El presupuesto, que contiene todos los costes asociados al proyecto a ejecutar.

Estructura de Descomposición del Trabajo



Cronograma del Trabajo



Presupuesto

Los costes para realización de este proyecto son los presentados a continuación. Se incluyen los materiales necesarios para el desarrollo de la aplicación, recogida de datos y la redacción del proyecto. Incluyendo el coste total de los recursos empleados en el proyecto obtenemos un presupuesto de 1.872,52€.

Coste de recursos materiales	
Descripción	Coste
Tablet	293,52 €
Diadema	600,00 €
Ordenador	949,00 €
TOTAL	1.842,52 €

Tabla 5 Coste material del proyecto imputando el total de los materiales

Dado que suponemos que algunos de estos materiales serán utilizados en más ocasiones además de este proyecto, se considera una estimación demasiado al alza imputar el coste total de los materiales a este único proyecto. Se calcula por tanto una amortización lineal a lo largo de la duración promedio de cada material. El cálculo en este escenario consiste en repartir el coste a lo largo de la vida útil de cada material, y considerar como coste el equivalente a la duración de este proyecto, 175 días, es decir 0,48 años. Con estos supuestos, el coste se reduce drásticamente a **128,16€**.

Coste de recursos materiales				
Descripción	Coste unitario	Amortización (años)	Duración del proyecto (años)	Coste
Tablet	293,52 €	4,00	0,48	35,22 €
Diadema	600,00 €	8,00	0,48	36,00 €
Ordenador	949,00 €	8,00	0,48	56,94 €
TOTAL				128,16 €

Tabla 6 Coste material del proyecto optimizado según una amortización lineal de los materiales

Bibliografía y referencias

- Banyard, P., Cassells, A., Green, P., Hartland, J., Hayes, N., & Reddy, P. (1995). *Introducción a los procesos cognitivos*. Buenos Aires: Ariel.
- Benítez-Burraco, A., & Murphy, E. (2018). The Oscillopathic Nature of Language Deficits in Autism: From Genes to Language Evolution. *frontiers in Human Neuroscience*, 1-17.
- Birbaumer, N., Kübler, A., Ghanayim, N., Hinterberger, T., Perelmouter, J., Kaise, J., . . . Flor, H. (Junio de 2000). The Thought Translation Device (TTD) for Completely Paralyzed Patients. *IEEE Transactions on Rehabilitation Engineering*, págs. 190-193.
- Bitbrain. (s.f.). Obtenido de <https://www.bitbrain.com/es>
- Blasi, A., & Roig, J. (s.f.). *¿Qué son las ondas Cerebrales?* Obtenido de NeuroFeedBack: <http://www.neurofeedback.cat/-iquetque-son-las-ondas-cerebrales-mnu-17>
- Braboszcz, C., Cahn, B., Levy, J., Fernandez, M., & Delorme, A. (24 de Enero de 2017). Increased Gamma Brainwave Amplitude Compared to Control in Three Different Meditation Traditions. *PLoS ONE*.
- Cahn, B., Levy, J., Fernandez, M., & Delorme, A. (24 de Enero de 2017). *Increased Gamma Brainwave Amplitude Compared to Control in Three Different Meditation Traditions*. Obtenido de PLOS ONE: <https://journals.plos.org/plosone/article?id=10.1371/journal.pone.0170647>
- Carboni, A., & Barg, G. (2016). En A. Vásquez Echevarría, *Manual de Introducción a la Psicología Cognitiva* (págs. 89-116). Montevideo: UdelaR.
- Choppin, A. (Agosto de 2000). EEG-Based Human Interface for Disabled Individuals: Emotion Expression with Neural Networks. Yokohama, Japón.
- Emotiv. (s.f.). Obtenido de <https://www.emotiv.com/>
- Gómez Cumpa, J., de la Cruz Vives, M., Herrera Ramírez, M., Martínez Velasco, J., González, F., Poggioli, L., . . . Ramirez Salguero, M. (2004). *Neurociencia Cognitiva y Educación*. Lambayeque: Fondo Editorial FACHSE.
- González-Castañeda, E. F., Torres-García, A. A., Reyes-García, C. A., & Villaseñor-Pineda, L. (2014). Sonificación de EEG para la clasificación de palabra no pronunciadas. *Research in Computing Science*, 61-72.
- González-Castañeda, E. F., Torres-García, A., Reyes-García, C., & Villaseñor-Pineda, L. (2016). Sonification and textification: Proposing methods for classifying unspoken words from EEG signals. *Biomedical Signal Processing and Control*.
- Guide. (s.f.). *Conoce tus Ondas Cerebrales: Alpha, Beta, Theta, Delta y Gamma*. Obtenido

- de NeuroEmocional: <https://neuroemocional.com/conoce-tus-ondas-cerebrales-alpha-beta-theta-delta-gamma/>
- Halo Sport*. (s.f.). Obtenido de <https://www.haloneuro.com/>
- Herrmann, N. (1990). *The creative brain*. North Carolina: Ned Herrmann Group.
- Isasi Viñuela, P., & Galván León, I. (2004). *Redes de neuronas artificiales: Un enfoque práctico*. Madrid: PEARSON EDUCACIÓN, S.A.
- Isasi Viñuela, P., & Galvan, I. M. (2004). *Redes de Neuronas Artificiales: Un Enfoque Práctico*. Pearson Educacion.
- Jara-Ettinger, J. (s.f.). *Ciencias cognitivas*. Obtenido de SaberMás: <https://www.sabermas.umich.mx/archivo/la-ciencia-en-pocas-palabras/18-numero-290/41-ciencias-cognitivas.html>
- Killingsworth, M., & Gilbert, D. (12 de Noviembre de 2010). *Science*. Obtenido de <http://www.sciencemag.org/>
- Kostov, A., & Polak, M. (Junio de 2000). Parallel Man-Machine Training in Development of EEG-Based Cursor Control. *IEEE Transactions on Rehabilitation Engineering*, págs. 203-205.
- Le Van Quyen, M., Martinerie, J., Navarro, V., Boon, P., D'Havé, M., Adam, C., . . . Baulac, M. (20 de Enero de 2001). Anticipation of epileptic seizures from standard EEG recordings. *The Lancet*, págs. 183-188.
- Llorente, A. (29 de Agosto de 2016). *¿Qué pasa en nuestro cerebro cuando leemos?* Obtenido de BBC: <https://www.bbc.com/mundo/noticias-36960389>
- Martín, G. F.-B. (2015). Procesos creativos en entornos musicales inmersivos; Un estudio de caso a través del análisis de metaversos. La Avatar Orchestra Metaverse en Second Life.
- Martín, G. F.-B. (s.f.). *Terapia Creativa Inmersiva (T.C.I.)M-000387/2017*. Registro Territorial de la Propiedad Intelectual.
- McGuire, P., Silbersweig, D., Murray, R., David, A., Frackowiak, R., & Frith, C. (1996). Functional anatomy of inner speech and auditory verbal imagery. *Psychological Medicine*, 29-38.
- Medical EXPO*. (s.f.). Obtenido de <http://www.medicaexpo.es/prod/medical-computer-systems/product-69255-565055.html>
- Mérida Perles, L. (2016-2017). *Las BCI (Brain Computer Interfaces): ¿magia o ciencia?* Zaragoza.
- MindWave Mobile: User Guide*. (Agosto de 2015). Obtenido de Neurosky: http://download.neurosky.com/support_page_files/MindWaveMobile/docs/mindw

ave_mobile_user_guide.pdf

Neurodidacta. (s.f.). Obtenido de http://www.neurodidacta.es/index.php?option=com_guru&view=gurutasks&catid=3&module=25-diagnostico&cid=130-rendimiento-diagn%C3%B3stico-del-eeg&tmpl=component&Itemid=0

Neuroelectrics. (s.f.). Obtenido de <https://www.neuroelectrics.com/>

Nieto Barrera, M. (2005). Afasias adquiridas: síndrome de Landau-Kleffer y otros. *Revista de neurología*, 123-129.

Norman, S. L., Dennison, M., Wolberht, E., Cramer, S. C., Srinivasan, R., & Reinkensmeyer, D. J. (2016). Movement Anticipation and EEG: Implications for BCI-Contingent Robot Therapy. *IEEE Transactions on Neural Systems and Rehabilitation Engineering*, págs. 911-919.

Pfurtscheller, G., Neuper, C., Flotzinger, D., & Pregenzer, M. (1997). EEG-based discrimination between imagination of right and left hand movement. *ELSEVIER*, 642-651.

Picard, A., Cheliout Heraut, F., Bouskraoui, M., Lemoine, M., Lacert, P., & Delattre, J. (2008). Sleep EEG and developmental dysphasia. *Developmental Medicine & Child Neurology*, 595-599.

Ruiz del Castillo, V., García Mateo, L., Abendea Alonso, L., & Segovia Pintos, S. (s.f.). *Trastorno Específico del Lenguaje (TEL)/Disfasia del desarrollo*. Obtenido de Luán Centro Logopédico: <http://www.logopedialuan.com/patologias/disfasia-de-desarrollo/>

Sánchez, J., Jurado, M., Carrasco, M., Ligonie, S., Pagano, J., Rivas, B., . . . Jofré Neila, M. (2017). Perfiles de actividad eléctrica cortical Beta 3 en tareas de cálculos matemáticos sencillos e imaginación de objetos. Un estudio descriptivo. *PSIUC*, 1-8.

Sanei, S., & Chambers, J. A. (2013). *Fundamentals of EEG Signal Processing*. Cardiff: John Wiley & Sons, Ltd.

Schooler, J., Reichle, E., & Halpern, D. (2004). Zoning out while reading: Evidence for dissociations between experience and metaconsciousness. En D. Levin, *Thinking and Seeing: Visual Metacognition in Adults and Children*. (págs. 203-226). Cambridge: MIT Press.

Schooler, J., Smallwood, J., Christoff, K., Handy, T., Reichle, E., & Sayette, M. (2011). Meta-awareness, perceptual decoupling and the wanderind mind. *Trends in Cognitive Sciences*, 319-326.

Torres García, A. A., Reyes García, C. A., & Villaseñor Pineda, L. (2011). Hacia la

clasificación de habla no pronunciada mediante electroencefalogramas (EEG). *XXXIV Congreso Nacional de Ingeniería Biomédica, Ixtapa-Zihuatanejo*. Guerrero, México.

Torres, F., Sánchez, C., & Palacio-Baus, K. (2014). Adquisición y análisis de señales cerebrales utilizando el dispositivo MindWave. *MASKANA*, 83-93.

Vallejo Ruiloba, J. (2015). *Introducción a la psicopatología y la psiquiatría*. Barcelona: MASSON.

Villena-González, M., López, V., & Rodríguez, E. (2016). Orienting attention to visual or verbal/auditory imagery differentially impairs the processing of visual stimuli. *NeuroImage*.

Viramonte, M. (2000). *Comprensión lectora. Dificultades estratégicas*. Buenos Aires: Editorial Trillas.

Vivar Vera, S. R., Abud Figueroa, M. A., Rodríguez Vásquez, G., Juárez Martínez, U., Peláez Camarena, G., Mercado Gutiérrez, J. A., & Gutiérrez Martínez, J. (2017). Desarrollo de una BCI utilizando el potencial P300 y la diadema Mindwave. *Research in Computing Science*, 140, 151-164.

Vygotsky, L. (1995). *Pensamiento y lenguaje*. Barcelona: Paidós.

Wikipedia. (2017). Obtenido de https://es.wikipedia.org/wiki/Hans_Berger

Wikipedia. (4 de Junio de 2018). Obtenido de https://es.wikipedia.org/wiki/Percept%C3%B3n_multicapa

Xiao-Mei, P., & Jeremy Hill, N. (2012). Silent Communication Toward Using Brain Signals. *IEEE PULSE*, 43-46.

Yuval-Greenberg, S., Tomer, O., Keren, A., Nelken, I., & Deouell, L. (8 de Mayo de 2008). Transient Induced Gamma-Band Response in EEG as a Manifestation of Miniature Saccades. *Neuron*, págs. 429-441.

ANEXOS

A.1. Código BCI

```
package com.neurosky.algo_sdk_sample;

import android.app.Activity;
import android.bluetooth.BluetoothAdapter;
import android.content.DialogInterface;
import android.content.pm.ActivityInfo;
import android.content.res.Configuration;
import android.graphics.Color;
import android.graphics.Paint;
import android.net.Uri;
import android.os.Build;
import android.os.Bundle;
import android.os.Environment;
import android.util.Log;
import android.view.View;
import android.widget.CheckBox;
import android.widget.EditText;
import android.widget.ImageView;
import android.widget.SeekBar;
import android.widget.TextView;
import android.widget.Button;
import android.widget.Toast;
import android.content.res.AssetManager;
import android.app.AlertDialog;

import java.io.BufferedWriter;
import java.io.File;
import java.io.FileReader;
import java.io.FileWriter;
import java.io.PrintWriter;
import java.util.Calendar;
import java.util.Timer;
import java.util.TimerTask;

import java.io.BufferedReader;
import java.io.IOException;
import java.io.InputStream;
import java.io.InputStreamReader;

import com.google.android.gms.appindexing.Action;
import com.google.android.gms.appindexing.AppIndex;
import com.google.android.gms.appindexing.Thing;
import com.google.android.gms.common.api.GoogleApiClient;
import com.neurosky.AlgoSdk.NskAlgoConfig;
import com.neurosky.AlgoSdk.NskAlgoDataType;
import com.neurosky.AlgoSdk.NskAlgoSdk;
import com.neurosky.AlgoSdk.NskAlgoSignalQuality;
import com.neurosky.AlgoSdk.NskAlgoState;
import com.neurosky.AlgoSdk.NskAlgoType;
import com.neurosky.connection.ConnectionStates;
```

```

import com.neurosky.connection.TgStreamHandler;
import com.neurosky.connection.TgStreamReader;
import com.neurosky.connection.DataType.MindDataType;

import com.androidplot.xy.*;

import static com.neurosky.algo_sdk_sample.R.id.dialog;

public class MainActivity extends Activity {

    final String TAG = "MainActivityTag";
    /*static {
        System.loadLibrary("NskAlgoAndroid");
    }*/

    // graph plot variables
    private final static int X_RANGE = 50;
    private SimpleXYSeries bp_deltaSeries = null;
    private SimpleXYSeries bp_thetaSeries = null;
    private SimpleXYSeries bp_alphaSeries = null;
    private SimpleXYSeries bp_betaSeries = null;
    private SimpleXYSeries bp_gammaSeries = null;

    // COMM SDK handles
    private TgStreamReader tgStreamReader;
    private BluetoothAdapter mBluetoothAdapter;

    // internal variables
    private boolean bInitied = false;
    private boolean bRunning = false;
    private NskAlgoType currentSelectedAlgo;

    // canned data variables
    private short raw_data[] = {0};
    private int raw_data_index = 0;
    private float output_data[];
    private int output_data_count = 0;
    private int raw_data_sec_len = 85;

    // UI components
    private XYPlot plot;
    private EditText text;

    private Button headsetButton;
    private Button cannedButton;
    private Button setAlgosButton;
    private Button setIntervalButton;
    private Button startButton;
    private Button stopButton;

    private SeekBar intervalSeekBar;
    private TextView intervalText;

    private Button bpText;

    private TextView attValue;
    private TextView medValue;

    private CheckBox attCheckBox;
    private CheckBox medCheckBox;
    private CheckBox blinkCheckBox;

```



```

private CheckBox bpCheckBox;

private TextView stateText;
private TextView sqText;

private TextView TitleDelta;
private TextView TitleGamma;
private TextView TitleTheta;
private TextView TitleAlpha;
private TextView TitleBeta;
private TextView TextDelta;
private TextView TextGamma;
private TextView TextTheta;
private TextView TextAlpha;
private TextView TextBeta;

private ImageView blinkImage;

private NskAlgoSdk nskAlgoSdk;

//variables cálculo promedio
int i = 1;
float sumaAlpha = 0;
float sumaDelta = 0;
float sumaTheta = 0;
float sumaBeta = 0;
float sumaGamma = 0;
float promedioAlpha = 0;
float promedioDelta = 0;
float promedioTheta = 0;
float promedioBeta = 0;
float promedioGamma = 0;

private int bLastOutputInterval = 1;
/**
 * ATTENTION: This was auto-generated to implement the App
Indexing API.
 * See https://g.co/AppIndexing/AndroidStudio for more
information.
 */
private GoogleApiClient client;

@Override
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    setContentView(R.layout.activity_main);

    nskAlgoSdk = new NskAlgoSdk();

    try {
        // (1) Make sure that the device supports Bluetooth and
Bluetooth is on
        mBluetoothAdapter = BluetoothAdapter.getDefaultAdapter();
        if (mBluetoothAdapter == null ||
!mBluetoothAdapter.isEnabled()) {
            Toast.makeText(
                this,
                "Please enable your Bluetooth and re-run this
program !",
                Toast.LENGTH_LONG).show();
            //finish();

```

```

    }
} catch (Exception e) {
    e.printStackTrace();
    Log.i(TAG, "error:" + e.getMessage());
    return;
}

headsetButton = (Button)
this.findViewById(R.id.headsetButton);
cannedButton = (Button)
this.findViewById(R.id.cannedDatabutton);
setAlgosButton = (Button)
this.findViewById(R.id.setAlgosButton);
setIntervalButton = (Button)
this.findViewById(R.id.setIntervalButton);
startButton = (Button) this.findViewById(R.id.startButton);
stopButton = (Button) this.findViewById(R.id.stopButton);

intervalSeekBar = (SeekBar)
this.findViewById(R.id.intervalSeekBar);
intervalText = (TextView)
this.findViewById(R.id.intervalText);

bpText = (Button) this.findViewById(R.id.bpTitle);

attValue = (TextView) this.findViewById(R.id.attText);
medValue = (TextView) this.findViewById(R.id.medText);

attCheckBox = (CheckBox) this.findViewById(R.id.attCheckBox);
medCheckBox = (CheckBox) this.findViewById(R.id.medCheckBox);
blinkCheckBox = (CheckBox)
this.findViewById(R.id.blinkCheckBox);
bpCheckBox = (CheckBox) this.findViewById(R.id.bpCheckBox);

blinkImage = (ImageView) this.findViewById(R.id.blinkImage);

stateText = (TextView) this.findViewById(R.id.stateText);
sqText = (TextView) this.findViewById(R.id.sqText);

TitleDelta = (TextView) this.findViewById(R.id.TitleDelta);
TitleGamma = (TextView) this.findViewById(R.id.TitleGamma);
TitleTheta = (TextView) this.findViewById(R.id.TitleTheta);
TitleAlpha = (TextView) this.findViewById(R.id.TitleAlpha);
TitleBeta = (TextView) this.findViewById(R.id.TitleBeta);
TextDelta = (TextView) this.findViewById(R.id.TextDelta);
TextGamma = (TextView) this.findViewById(R.id.TextGamma);
TextTheta = (TextView) this.findViewById(R.id.TextTheta);
TextAlpha = (TextView) this.findViewById(R.id.TextAlpha);
TextBeta = (TextView) this.findViewById(R.id.TextBeta);

headsetButton.setOnClickListener(new View.OnClickListener() {
    @Override
    public void onClick(View v) {
        output_data_count = 0;
        output_data = null;

        raw_data = new short[512];
        raw_data_index = 0;

        cannedButton.setEnabled(false);
    }
});

```

```

        headsetButton.setEnabled(false);

        startButton.setEnabled(false);

        // Example of constructor public
        TgStreamReader(BluetoothAdapter ba, TgStreamHandler tgStreamHandler)
        tgStreamReader = new TgStreamReader(mBluetoothAdapter,
callback);

        if (tgStreamReader != null &&
tgStreamReader.isBTConnected()) {

            // Prepare for connecting
            tgStreamReader.stop();
            tgStreamReader.close();
        }

        // (4) Demo of using connect() and start() to replace
connectAndStart(),
        // please call start() when the state is changed to
STATE_CONNECTED
        tgStreamReader.connect();
    }
});

cannedButton.setOnClickListener(new View.OnClickListener() {
    @Override
    public void onClick(View v) {
        output_data_count = 0;
        output_data = null;

        System.gc();

        headsetButton.setEnabled(false);
        cannedButton.setEnabled(false);

        AssetManager assetManager = getAssets();
        InputStream inputStream = null;

        Log.d(TAG, "Reading output data");
        try {
            int j;
            // check the output count first
            inputStream =
assetManager.open("output_data.bin");
            output_data_count = 0;
            BufferedReader reader = new BufferedReader(new
InputStreamReader(inputStream));
            try {
                String line = reader.readLine();
                while (!(line == null || line.isEmpty())) {
                    output_data_count++;
                    line = reader.readLine();
                }
            } catch (IOException e) {

            }
            inputStream.close();

            if (output_data_count > 0) {
                inputStream =

```

```

assetManager.open("output_data.bin");
        output_data = new float[output_data_count];
        //ap = new float[output_data_count];
        j = 0;
        reader = new BufferedReader(new
InputStreamReader(inputStream));
        try {
            String line = reader.readLine();
            while (j < output_data_count) {
                output_data[j++] =
Float.parseFloat(line);
                line = reader.readLine();
            }
        } catch (IOException e) {

        }
        inputStream.close();
    }
} catch (IOException e) {
}

Log.d(TAG, "Reading raw data");
try {
    inputStream =
assetManager.open("raw_data_em.bin");
    raw_data = readData(inputStream, 512 *
raw_data_sec_len);
    raw_data_index = 512 * raw_data_sec_len;
    inputStream.close();

    nskAlgoSdk.NskAlgoDataStream(NskAlgoDataType.NSK_ALGO_DATA_TYPE_BULK_E
EG.value, raw_data, 512 * raw_data_sec_len);
} catch (IOException e) {

}

Log.d(TAG, "Finished reading data");
});

startButton.setOnClickListener(new View.OnClickListener() {
    @Override
    public void onClick(View v) {
        if (bRunning == false) {
            nskAlgoSdk.NskAlgoStart(false);
        } else {
            nskAlgoSdk.NskAlgoPause();
        }
        File fichero = new
File(Environment.getExternalStorageDirectory().getPath() +
"/OndasCerebrales/OndasCerebrales.txt");
        VaciaFichero(fichero);
    }
});

stopButton.setOnClickListener(new View.OnClickListener() {
    @Override
    public void onClick(View v) {
        nskAlgoSdk.NskAlgoStop();
    }
});

```

```

        setAlgosButton.setOnClickListener(new View.OnClickListener() {
            @Override
            public void onClick(View v) {
                // check selected algos
                int algoTypes = 0; // =
NskAlgoType.NSK_ALGO_TYPE_CR.value;

                startButton.setEnabled(false);
                stopButton.setEnabled(false);
                clearAllSeries();
                text.setVisibility(View.INVISIBLE);
                text.setText("");

                bpText.setEnabled(false);

                currentSelectedAlgo =
NskAlgoType.NSK_ALGO_TYPE_INVALID;
                intervalSeekBar.setEnabled(false);
                setIntervalButton.setEnabled(false);
                intervalText.setText("--");

                attValue.setText("--");
                medValue.setText("--");

                stateText.setText("");
                sqText.setText("");

                if (medCheckBox.isChecked()) {
                    algoTypes += NskAlgoType.NSK_ALGO_TYPE_MED.value;
                }
                if (attCheckBox.isChecked()) {
                    algoTypes += NskAlgoType.NSK_ALGO_TYPE_ATT.value;
                }
                if (blinkCheckBox.isChecked()) {
                    algoTypes +=
NskAlgoType.NSK_ALGO_TYPE_BLINK.value;
                }
                if (bpCheckBox.isChecked()) {
                    algoTypes += NskAlgoType.NSK_ALGO_TYPE_BP.value;
                    bpText.setEnabled(true);
                    bp_deltaSeries = createSeries("Delta");
                    bp_thetaSeries = createSeries("Theta");
                    bp_alphaSeries = createSeries("Alpha");
                    bp_betaSeries = createSeries("Beta");
                    bp_gammaSeries = createSeries("Gamma");
                }

                if (algoTypes == 0) {
                    showDialog("Please select at least one
algorithm");
                } else {
                    if (bInited) {
                        nskAlgoSdk.NskAlgoUninit();
                        bInited = false;
                    }
                    int ret = nskAlgoSdk.NskAlgoInit(algoTypes,
getFilesDir().getAbsolutePath());
                    if (ret == 0) {
                        bInited = true;

```

```

    }

    Log.d(TAG, "NSK_ALGO_Init() " + ret);
    String sdkVersion = "SDK ver.: " +
nskAlgoSdk.NskAlgoSdkVersion();

    if ((algoTypes &
NskAlgoType.NSK_ALGO_TYPE_ATT.value) != 0) {
        sdkVersion += "\nATT ver.: " +
nskAlgoSdk.NskAlgoAlgoVersion(NskAlgoType.NSK_ALGO_TYPE_ATT.value);
    }
    if ((algoTypes &
NskAlgoType.NSK_ALGO_TYPE_MED.value) != 0) {
        sdkVersion += "\nMED ver.: " +
nskAlgoSdk.NskAlgoAlgoVersion(NskAlgoType.NSK_ALGO_TYPE_MED.value);
    }
    if ((algoTypes &
NskAlgoType.NSK_ALGO_TYPE_BLINK.value) != 0) {
        sdkVersion += "\nBlink ver.: " +
nskAlgoSdk.NskAlgoAlgoVersion(NskAlgoType.NSK_ALGO_TYPE_BLINK.value);
    }
    if ((algoTypes &
NskAlgoType.NSK_ALGO_TYPE_BP.value) != 0) {
        sdkVersion += "\nEEG Bandpower ver.: " +
nskAlgoSdk.NskAlgoAlgoVersion(NskAlgoType.NSK_ALGO_TYPE_BP.value);
    }
    showToast(sdkVersion, Toast.LENGTH_LONG);
}
});

bpText.setEnabled(false);
bpText.setOnClickListener(new View.OnClickListener() {
    @Override
    public void onClick(View view) {
        removeAllSeriesFromPlot();
        setupPlot(-20, 20, "EEG Bandpower");
        addSeries(plot, bp_deltaSeries,
R.xml.line_point_formatter_with_plf1);
        addSeries(plot, bp_thetaSeries,
R.xml.line_point_formatter_with_plf2);
        addSeries(plot, bp_alphaSeries,
R.xml.line_point_formatter_with_plf3);
        addSeries(plot, bp_betaSeries,
R.xml.line_point_formatter_with_plf4);
        addSeries(plot, bp_gammaSeries,
R.xml.line_point_formatter_with_plf5);
        plot.redraw();

        text.setVisibility(View.INVISIBLE);

        currentSelectedAlgo = NskAlgoType.NSK_ALGO_TYPE_BP;

        intervalSeekBar.setMax(1);
        intervalSeekBar.setProgress(0);
        intervalSeekBar.setEnabled(false);
        intervalText.setText(String.format("%d", 1));
        setIntervalButton.setEnabled(false);
    }
});

```

```

        intervalSeekBar.setEnabled(false);
        intervalSeekBar.setOnSeekBarChangeListener(new
SeekBar.OnSeekBarChangeListener() {
            @Override
            public void onProgressChanged(SeekBar seekBar, int
progress, boolean fromUser) {
                }

            @Override
            public void onStartTrackingTouch(SeekBar seekBar) {
                bLastOutputInterval = seekBar.getProgress();
            }

            @Override
            public void onStopTrackingTouch(SeekBar seekBar) {
                }
        });

        setIntervalButton.setEnabled(false);
        setIntervalButton.setOnClickListener(new
View.OnClickListener() {
            @Override
            public void onClick(View v) {
                int ret = -1;
                String toastStr = "";

                if (ret == 0) {
                    showToast(toastStr + ": success",
Toast.LENGTH_SHORT);
                } else {
                    showToast(toastStr + ": fail",
Toast.LENGTH_SHORT);
                }
            }
        });

        nskAlgoSdk.setOnSignalQualityListener(new
NskAlgoSdk.OnSignalQualityListener() {
            @Override
            public void onSignalQuality(int level) {
                //Log.d(TAG, "NskAlgoSignalQualityListener: level: " +
level);

                final int fLevel = level;
                runOnUiThread(new Runnable() {
                    @Override
                    public void run() {
                        // change UI elements here
                        String sqStr =
NskAlgoSignalQuality.values()[fLevel].toString();
                        sqText.setText(sqStr);
                    }
                });
            }
        });

        nskAlgoSdk.setOnStateChangeListener(new
NskAlgoSdk.OnStateChangeListener() {
            @Override
            public void onStateChange(int state, int reason) {
                String stateStr = "";
                String reasonStr = "";

```

```

        for (NskAlgoState s : NskAlgoState.values()) {
            if (s.value == state) {
                stateStr = s.toString();
            }
        }
        for (NskAlgoState r : NskAlgoState.values()) {
            if (r.value == reason) {
                reasonStr = r.toString();
            }
        }
        Log.d(TAG, "NskAlgoSdkStateChangeListener: state: " +
stateStr + ", reason: " + reasonStr);
        final String finalStateStr = stateStr + " | " +
reasonStr;

        final int finalState = state;
        runOnUiThread(new Runnable() {
            @Override
            public void run() {
                // change UI elements here
                stateText.setText(finalStateStr);

                if (finalState ==
NskAlgoState.NSK_ALGO_STATE_RUNNING.value || finalState ==
NskAlgoState.NSK_ALGO_STATE_COLLECTING_BASELINE_DATA.value) {
                    bRunning = true;
                    startButton.setText("Pause");
                    startButton.setEnabled(true);
                    stopButton.setEnabled(true);
                } else if (finalState ==
NskAlgoState.NSK_ALGO_STATE_STOP.value) {
                    bRunning = false;
                    raw_data = null;
                    raw_data_index = 0;
                    startButton.setText("Start");
                    startButton.setEnabled(true);
                    stopButton.setEnabled(false);

                    headsetButton.setEnabled(true);
                    cannedButton.setEnabled(true);

                    if (tgStreamReader != null &&
tgStreamReader.isBTConnected()) {

                        // Prepare for connecting
                        tgStreamReader.stop();
                        tgStreamReader.close();
                    }

                    output_data_count = 0;
                    output_data = null;

                    System.gc();
                } else if (finalState ==
NskAlgoState.NSK_ALGO_STATE_PAUSE.value) {
                    bRunning = false;
                    startButton.setText("Start");
                    startButton.setEnabled(true);
                    stopButton.setEnabled(true);
                } else if (finalState ==
NskAlgoState.NSK_ALGO_STATE_ANALYSING_BULK_DATA.value) {
                    bRunning = true;

```



```

        startButton.setText("Start");
        startButton.setEnabled(false);
        stopButton.setEnabled(true);
    } else if (finalState ==
NskAlgoState.NSK_ALGO_STATE_INITED.value || finalState ==
NskAlgoState.NSK_ALGO_STATE_UNINTIED.value) {
        bRunning = false;
        startButton.setText("Start");
        startButton.setEnabled(true);
        stopButton.setEnabled(false);
    }
    }
    });
}
});

    nskAlgoSdk.setOnSignalQualityListener(new
NskAlgoSdk.OnSignalQualityListener() {
        @Override
        public void onSignalQuality(final int level) {
            runOnUiThread(new Runnable() {
                @Override
                public void run() {
                    // change UI elements here
                    String sqStr =
NskAlgoSignalQuality.values()[level].toString();
                    sqText.setText(sqStr);
                }
            });
        }
    });

    nskAlgoSdk.setOnBPAlgoIndexListener(new
NskAlgoSdk.OnBPAlgoIndexListener() {
        @Override
        public void onBPAlgoIndex(float delta, float theta, float
alpha, float beta, float gamma) {
            Log.d(TAG, "NskAlgoBPAlgoIndexListener: BP: D[" +
delta + " dB] T[" + theta + " dB] A[" + alpha + " dB] B[" + beta + "
dB] G[" + gamma + " dB]");

            final float fDelta = delta, fTheta = theta, fAlpha =
alpha, fBeta = beta, fGamma = gamma;
            runOnUiThread(new Runnable() {
                @Override
                public void run() {
                    // change UI elements here
                    AddValueToPlot(bp_deltaSeries, fDelta);
                    AddValueToPlot(bp_thetaSeries, fTheta);
                    AddValueToPlot(bp_alphaSeries, fAlpha);
                    AddValueToPlot(bp_betaSeries, fBeta);
                    AddValueToPlot(bp_gammaSeries, fGamma);

                    File fichero = new
File(Environment.getExternalStorageDirectory().getPath() +
"/OndasCerebrales/OndasCerebrales.txt");

                    TextAlpha.setText(" " +
Float.toString(fAlpha));

```

```

        TextDelta.setText(" " +
Float.toString(fDelta));
        TextGamma.setText(" " +
Float.toString(fGamma));
        TextTheta.setText(" " +
Float.toString(fTheta));
        TextBeta.setText(" " +
Float.toString(fBeta));
        //ESCRITURA EN EL FICHERO
        EscribeDatos(fichero, fAlpha, fDelta, fGamma,
fTheta, fBeta);
    }
    });
}
});

nskAlgoSdk.setOnAttAlgoIndexListener(new
NskAlgoSdk.OnAttAlgoIndexListener() {
    @Override
    public void onAttAlgoIndex(int value) {
        Log.d(TAG, "NskAlgoAttAlgoIndexListener: Attention:" +
value);

        String attStr = "[" + value + "]";
        final String finalAttStr = attStr;
        runOnUiThread(new Runnable() {
            @Override
            public void run() {
                // change UI elements here
                attValue.setText(finalAttStr);
            }
        });
    }
});

nskAlgoSdk.setOnMedAlgoIndexListener(new
NskAlgoSdk.OnMedAlgoIndexListener() {
    @Override
    public void onMedAlgoIndex(int value) {
        Log.d(TAG, "NskAlgoMedAlgoIndexListener: Meditation:"
+ value);

        String medStr = "[" + value + "]";
        final String finalMedStr = medStr;
        runOnUiThread(new Runnable() {
            @Override
            public void run() {
                // change UI elements here
                medValue.setText(finalMedStr);
            }
        });
    }
});

nskAlgoSdk.setOnEyeBlinkDetectionListener(new
NskAlgoSdk.OnEyeBlinkDetectionListener() {
    @Override
    public void onEyeBlinkDetect(int strength) {
        Log.d(TAG, "NskAlgoEyeBlinkDetectionListener: Eye
blink detected: " + strength);

```

```

        runOnUiThread(new Runnable() {
            @Override
            public void run() {
                blinkImage.setImageResource(R.mipmap.led_on);
                Timer timer = new Timer();

                timer.schedule(new TimerTask() {
                    public void run() {
                        runOnUiThread(new Runnable() {
                            @Override
                            public void run() {
                                blinkImage.setImageResource(R.mipmap.led_off);
                            }
                        });
                    }
                }, 500);
            }
        });

        // initialize our XYPlot reference:
        plot = (XYPlot) findViewById(R.id.myPlot);
        plot.setVisibility(View.INVISIBLE);
        text = (EditText) findViewById(R.id.myText);
        text.setVisibility(View.INVISIBLE);
        // ATTENTION: This was auto-generated to implement the App
        Indexing API.
        // See https://g.co/AppIndexing/AndroidStudio for more
        information.
        client = new
        GoogleApiClient.Builder(this).addApi(AppIndex.API).build();
    }

    void VaciaFichero(File fichero){
        FileWriter bw = null;

        try{
            bw = new FileWriter(fichero, false);
            bw.write("ALPHA\t\t\t");
            bw.write("DELTA\t\t\t");
            bw.write("GAMMA\t\t\t");
            bw.write("THETA\t\t\t");
            bw.write("BETA\n");
            bw.write("\n");

            bw.close();
        } catch (Exception e2) {
            e2.printStackTrace();
        }
    }

    void EscribirDatos(File fichero, float dato1, float dato2, float
    dato3, float dato4, float dato5) {

        FileWriter bw = null;
        try {

```

```

        bw = new FileWriter(fichero, true);
        bw.write(Float.toString(dato1)+"\t\t");
        bw.write(Float.toString(dato2)+"\t\t");
        bw.write(Float.toString(dato3)+"\t\t");
        bw.write(Float.toString(dato4)+"\t\t");
        bw.write(Float.toString(dato5)+"\n");

        ;

        bw.close();
    } catch (Exception e2) {
        e2.printStackTrace();
    }

}

private void removeAllSeriesFromPlot() {
    if (bp_deltaSeries != null) {
        plot.removeSeries(bp_deltaSeries);
    }
    if (bp_thetaSeries != null) {
        plot.removeSeries(bp_thetaSeries);
    }
    if (bp_alphaSeries != null) {
        plot.removeSeries(bp_alphaSeries);
    }
    if (bp_betaSeries != null) {
        plot.removeSeries(bp_betaSeries);
    }
    if (bp_gammaSeries != null) {
        plot.removeSeries(bp_gammaSeries);
    }
    System.gc();
}

private void clearAllSeries() {
    if (bp_deltaSeries != null) {
        plot.removeSeries(bp_deltaSeries);
        bp_deltaSeries = null;
    }
    if (bp_thetaSeries != null) {
        plot.removeSeries(bp_thetaSeries);
        bp_thetaSeries = null;
    }
    if (bp_alphaSeries != null) {
        plot.removeSeries(bp_alphaSeries);
        bp_alphaSeries = null;
    }
    if (bp_betaSeries != null) {
        plot.removeSeries(bp_betaSeries);
        bp_betaSeries = null;
    }
    if (bp_gammaSeries != null) {
        plot.removeSeries(bp_gammaSeries);
        bp_gammaSeries = null;
    }
    plot.setVisibility(View.INVISIBLE);
    System.gc();
}

```

```

    }

    private XYPlot setupPlot(Number rangeMin, Number rangeMax, String
title) {
        // initialize our XYPlot reference:
        plot = (XYPlot) findViewById(R.id.myPlot);

        plot.setDomainLeftMax(0);
        plot.setDomainRightMin(X_RANGE);
        plot.setDomainRightMax(X_RANGE);

        if ((rangeMax.intValue() - rangeMin.intValue()) < 10) {
            plot.setRangeStepValue((rangeMax.intValue() -
rangeMin.intValue() + 1));
        } else {
            plot.setRangeStepValue(11);
        }
        plot.setRangeBoundaries(rangeMin.intValue(),
rangeMax.intValue(), BoundaryMode.FIXED);

        plot.getGraphWidget().getGridBackgroundPaint().setColor(Color.WHITE);

        plot.setTicksPerDomainLabel(10);
        plot.getGraphWidget().setDomainLabelOrientation(-45);

        plot.setPlotPadding(0, 0, 0, 0);
        plot.setTitle(title);

        plot.setVisibility(View.VISIBLE);

        return plot;
    }

    private SimpleXYSeries createSeries(String seriesName) {
        // Turn the above arrays into XYSeries':
        SimpleXYSeries series = new SimpleXYSeries(
            null, // SimpleXYSeries takes a List so turn
our array into a List
            SimpleXYSeries.ArrayFormat.Y_VALS_ONLY, // Y_VALS_ONLY
means use the element index as the x value
            seriesName); // Set the
display title of the series

        series.useImplicitXVals();

        return series;
    }

    private SimpleXYSeries addSeries(XYPlot plot, SimpleXYSeries
series, int formatterId) {

        // Create a formatter to use for drawing a series using
LineAndPointRenderer
        // and configure it from xml:
        LineAndPointFormatter seriesFormat = new
LineAndPointFormatter();
        seriesFormat.setPointLabelFormatter(null);
        seriesFormat.configure(getApplicationContext(), formatterId);
        seriesFormat.setVertexPaint(null);
        series.useImplicitXVals();
    }

```

```

        // add a new series' to the xyplot:
        plot.addSeries(series, seriesFormat);

        return series;
    }

    private int gcCount = 0;

    private void AddValueToPlot(SimpleXYSeries series, float value) {
        if (series.size() >= X_RANGE) {
            series.removeFirst();
        }
        Number num = value;
        series.addLast(null, num);
        plot.redraw();
        gcCount++;
        if (gcCount >= 20) {
            System.gc();
            gcCount = 0;
        }
    }

    private short[] readData(InputStream is, int size) {
        short data[] = new short[size];
        int lineCount = 0;
        BufferedReader reader = new BufferedReader(new
InputStreamReader(is));
        try {
            while (lineCount < size) {
                String line = reader.readLine();
                if (line == null || line.isEmpty()) {
                    Log.d(TAG, "lineCount=" + lineCount);
                    break;
                }
                data[lineCount] = Short.parseShort(line);
                lineCount++;
            }
            Log.d(TAG, "lineCount=" + lineCount);
        } catch (IOException e) {

        }
        return data;
    }

    @Override
    public void onBackPressed() {
        nskAlgoSdk.NskAlgoUninit();
        finish();
    }

    @Override
    public void onConfigurationChanged(Configuration newConfig) {
        super.onConfigurationChanged(newConfig);
    }

    setRequestedOrientation(ActivityInfo.SCREEN_ORIENTATION_PORTRAIT);
}

    public static String Datetime() {
        Calendar c = Calendar.getInstance();

```

```

        String sDate = "[" + c.get(Calendar.YEAR) + "/"
            + (c.get(Calendar.MONTH) + 1)
            + "/" + c.get(Calendar.DAY_OF_MONTH)
            + " " + c.get(Calendar.HOURLY_OF_DAY)
            + ":" + String.format("%02d", c.get(Calendar.MINUTE))
            + ":" + String.format("%02d", c.get(Calendar.SECOND))
+ "]"
        return sDate;
    }

    private TgStreamHandler callback = new TgStreamHandler() {

        @Override
        public void onStatesChanged(int connectionStates) {
            // TODO Auto-generated method stub
            Log.d(TAG, "connectionStates change to: " +
connectionStates);
            switch (connectionStates) {
                case ConnectionStates.STATE_CONNECTING:
                    // Do something when connecting
                    break;
                case ConnectionStates.STATE_CONNECTED:
                    // Do something when connected
                    tgStreamReader.start();
                    showToast("Connected", Toast.LENGTH_SHORT);
                    break;
                case ConnectionStates.STATE_WORKING:
                    // Do something when working

                    // (9) demo of recording raw data , stop() will
                    call stopRecordRawData,
                    // or you can add a button to control it.
                    // You can change the save path by calling
                    setRecordStreamFilePath(String filePath) before startRecordRawData
                    // tgStreamReader.startRecordRawData();

                    MainActivity.this.runOnUiThread(new Runnable() {
                        public void run() {
                            Button startButton = (Button)
findViewById(R.id.startButton);
                            startButton.setEnabled(true);
                        }
                    });

                    break;
                case ConnectionStates.STATE_GET_DATA_TIME_OUT:
                    // Do something when getting data timeout

                    // (9) demo of recording raw data, exception
                    handling
                    // tgStreamReader.stopRecordRawData();

                    showToast("Get data time out!",
Toast.LENGTH_SHORT);

                    if (tgStreamReader != null &&
tgStreamReader.isBTConnected()) {
                        tgStreamReader.stop();
                        tgStreamReader.close();
                    }
            }
        }
    }

```

```

        break;
    case ConnectionStates.STATE_STOPPED:
        // Do something when stopped
        // We have to call tgStreamReader.stop() and
        tgStreamReader.close() much more than
        // tgStreamReader.connectAndstart(), because we
        have to prepare for that.

        break;
    case ConnectionStates.STATE_DISCONNECTED:
        // Do something when disconnected
        break;
    case ConnectionStates.STATE_ERROR:
        // Do something when you get error message
        break;
    case ConnectionStates.STATE_FAILED:
        // Do something when you get failed message
        // It always happens when open the BluetoothSocket
        error or timeout

        // Maybe the device is not working normal.
        // Maybe you have to try again
        break;
    }
}

@Override
public void onRecordFail(int flag) {
    // You can handle the record error message here
    Log.e(TAG, "onRecordFail: " + flag);
}

@Override
public void onChecksumFail(byte[] payload, int length, int
checksum) {
    // You can handle the bad packets here.
}

@Override
public void onDataReceived(int datatype, int data, Object obj)
{
    // You can handle the received data here
    // You can feed the raw data to algo sdk here if
    necessary.
    //Log.i(TAG, "onDataReceived");
    switch (datatype) {
        case MindDataType.CODE_ATTENTION:
            short attValue[] = {(short) data};

            nskAlgoSdk.NskAlgoDataStream(NskAlgoDataType.NSK_ALGO_DATA_TYPE_ATT.va
lue, attValue, 1);
            break;
        case MindDataType.CODE_MEDITATION:
            short medValue[] = {(short) data};

            nskAlgoSdk.NskAlgoDataStream(NskAlgoDataType.NSK_ALGO_DATA_TYPE_MED.va
lue, medValue, 1);
            break;
        case MindDataType.CODE_POOR_SIGNAL:
            short pqValue[] = {(short) data};

```



```

nskAlgoSdk.NskAlgoDataStream(NskAlgoDataType.NSK_ALGO_DATA_TYPE_PQ.value, pqValue, 1);
        break;
    case MindDataType.CODE_RAW:
        raw_data[raw_data_index++] = (short) data;
        if (raw_data_index == 512) {
            nskAlgoSdk.NskAlgoDataStream(NskAlgoDataType.NSK_ALGO_DATA_TYPE_EEG.value, raw_data, raw_data_index);
            raw_data_index = 0;
        }
        break;
    default:
        break;
    }
}

};

public void showToast(final String msg, final int timeStyle) {
    MainActivity.this.runOnUiThread(new Runnable() {
        public void run() {
            Toast.makeText(getApplicationContext(), msg,
timeStyle).show();
        }
    });
}

private void showDialog(String message) {
    new AlertDialog.Builder(this)
        .setTitle("")
        .setMessage(message)
        .setPositiveButton(android.R.string.yes, new
DialogInterface.OnClickListener() {
            public void onClick(DialogInterface dialog, int
which) {
                // continue with delete
            }
        })
        .setIcon(android.R.drawable.ic_dialog_alert)
        .show();
}

/**
 * ATTENTION: This was auto-generated to implement the App
Indexing API.
 * See https://g.co/AppIndexing/AndroidStudio for more
information.
 */
public Action getIndexApiAction() {
    Thing object = new Thing.Builder()
        .setName("Main Page") // TODO: Define a title for the
content shown.
        // TODO: Make sure this auto-generated URL is correct.
        .setUrl(Uri.parse("http://[ENTER-YOUR-URL-HERE]"))
        .build();
    return new Action.Builder(Action.TYPE_VIEW)
        .setObject(object)
        .setActionStatus(Action.STATUS_TYPE_COMPLETED)

```

```

        .build();
    }

    @Override
    public void onStart() {
        super.onStart();

        // ATTENTION: This was auto-generated to implement the App
        Indexing API.
        // See https://g.co/AppIndexing/AndroidStudio for more
        information.
        client.connect();
        AppIndex.AppIndexApi.start(client, getIndexApiAction());
    }

    @Override
    public void onStop() {
        super.onStop();

        // ATTENTION: This was auto-generated to implement the App
        Indexing API.
        // See https://g.co/AppIndexing/AndroidStudio for more
        information.
        AppIndex.AppIndexApi.end(client, getIndexApiAction());
        client.disconnect();
    }
    /**
     * Crea un diálogo de alerta sencillo
     * @return Nuevo diálogo
     */
}

```